



Rest Hippo

User Guide

Version 1.0.0 · July 7, 2026

Contents

1 Overview

2 Getting Started

3 Collections & the Tree

4 Building Requests

5 Authentication

6 Variables & Environments

7 Functions

8 GraphQL

9 WebSockets

10 Reading Responses

11 Scripts

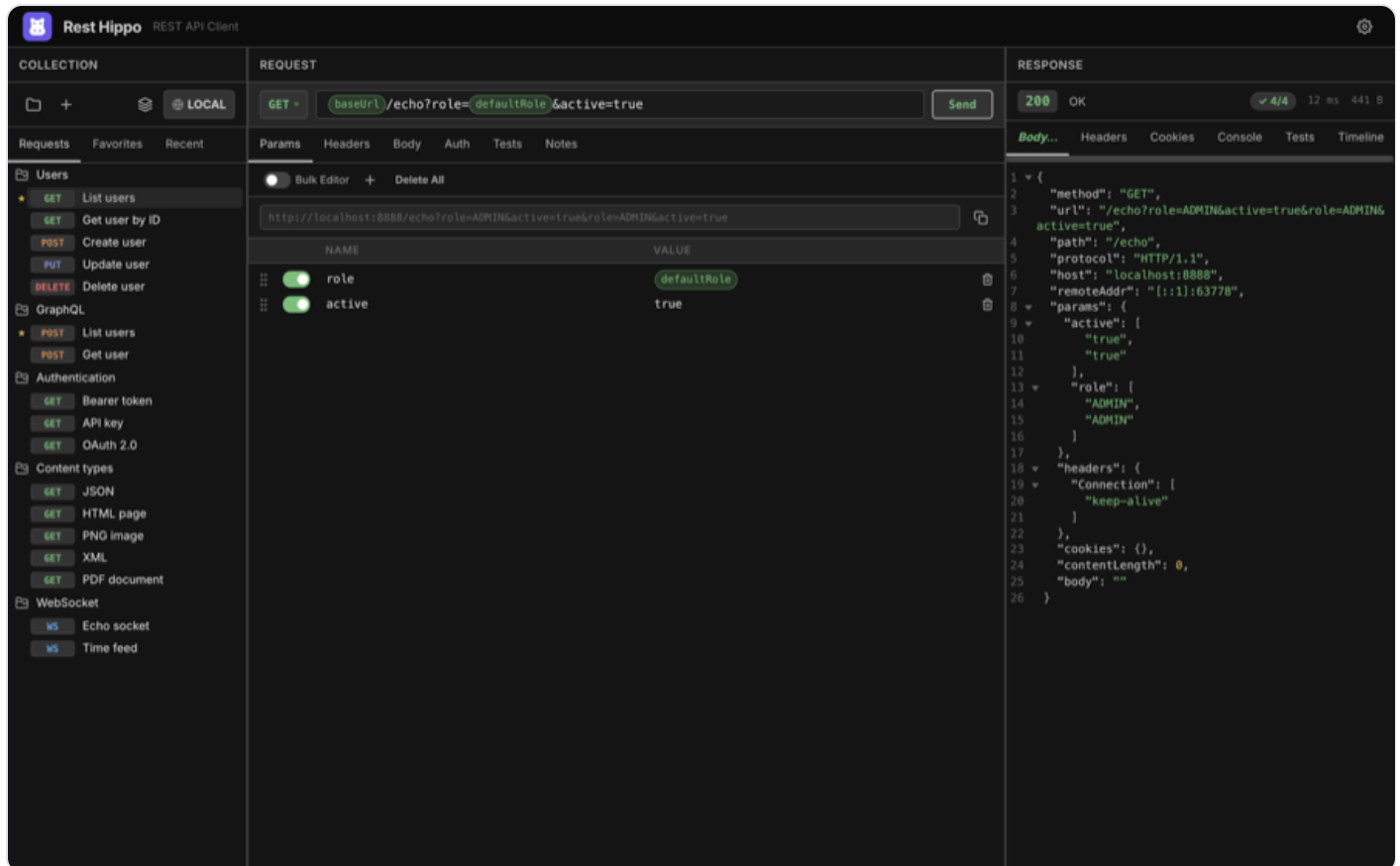
12 Import, Export & Backup

13 Settings & Themes

14 Keyboard Shortcuts

Rest Hippo User Guide

Rest Hippo is a fast, cross-platform desktop REST API client — like Postman or Insomnia — for building, sending, and inspecting HTTP, GraphQL, and WebSocket requests. It runs natively (no browser CORS limits), stores everything in local files, and ships with a built-in theme editor and a bundled UI font.



This guide walks through everything Rest Hippo can do, from sending your first request to OAuth 2.0, environments, GraphQL schema introspection, and encrypted backups.

Contents

Page	What's inside
Getting Started	Install, the three-panel layout, and sending your first request
Collections & the Tree	Organizing requests into collections and folders; Favorites and Recent
Building Requests	Methods, the URL bar, query params, headers, and body editors
Authentication	API Key, Basic, Bearer, Digest, NTLM, AWS SigV4, and OAuth 2.0
Variables & Environments	<code>{{variables}}</code> , environments, collection/folder scopes, and response captures
Functions	Built-in <code>{{ }}</code> functions: UUIDs, timestamps, encoding, request outputs, and <code>run</code>
GraphQL	The query/variables editor, schema introspection, and validation
WebSockets	Connecting, sending frames, and reading the frame log
Reading Responses	Body rendering, previews, headers, cookies, timeline, and search
Scripts	Pre-request & after-response JavaScript with the <code>hippo.*</code> API
Import, Export & Backup	Postman/Insomnia/OpenAPI/HAR and whole-workspace backups
Settings & Themes	Appearance, layouts, proxy, retries, and the theme editor
Keyboard Shortcuts	Every shortcut in one place

A quick tour

Rest Hippo's window has three panels:

- **Collections** (left) — your saved requests, organized into collections and folders, plus the **Favorites** and **Recent** tabs. Its toolbar holds the **collection** and **environment** selectors.
- **Request** (center) — where you set the method, URL, query parameters, headers, body, authentication, and post-response captures.
- **Response** (right) — the status, timing, body (syntax-highlighted), headers, cookies, console output, and a request timeline.

At the top right is **Settings**. From there you can rearrange the three panels into four different [layouts](#) and restyle everything with [themes](#).

The screenshots in this guide use a demo collection called **Demo API** and a **Local** environment pointing at a test server. Yours will show your own collections.

Ready? Start with [Getting Started](#).

Getting Started

[← Back to contents](#)

Installing

Rest Hippo ships as a native desktop app for **macOS**, **Windows**, and **Linux**. Download the installer for your platform, run it, and launch Rest Hippo — there's nothing else to set up. Your data lives in a local folder under your user profile, so requests, environments, and settings persist between sessions.

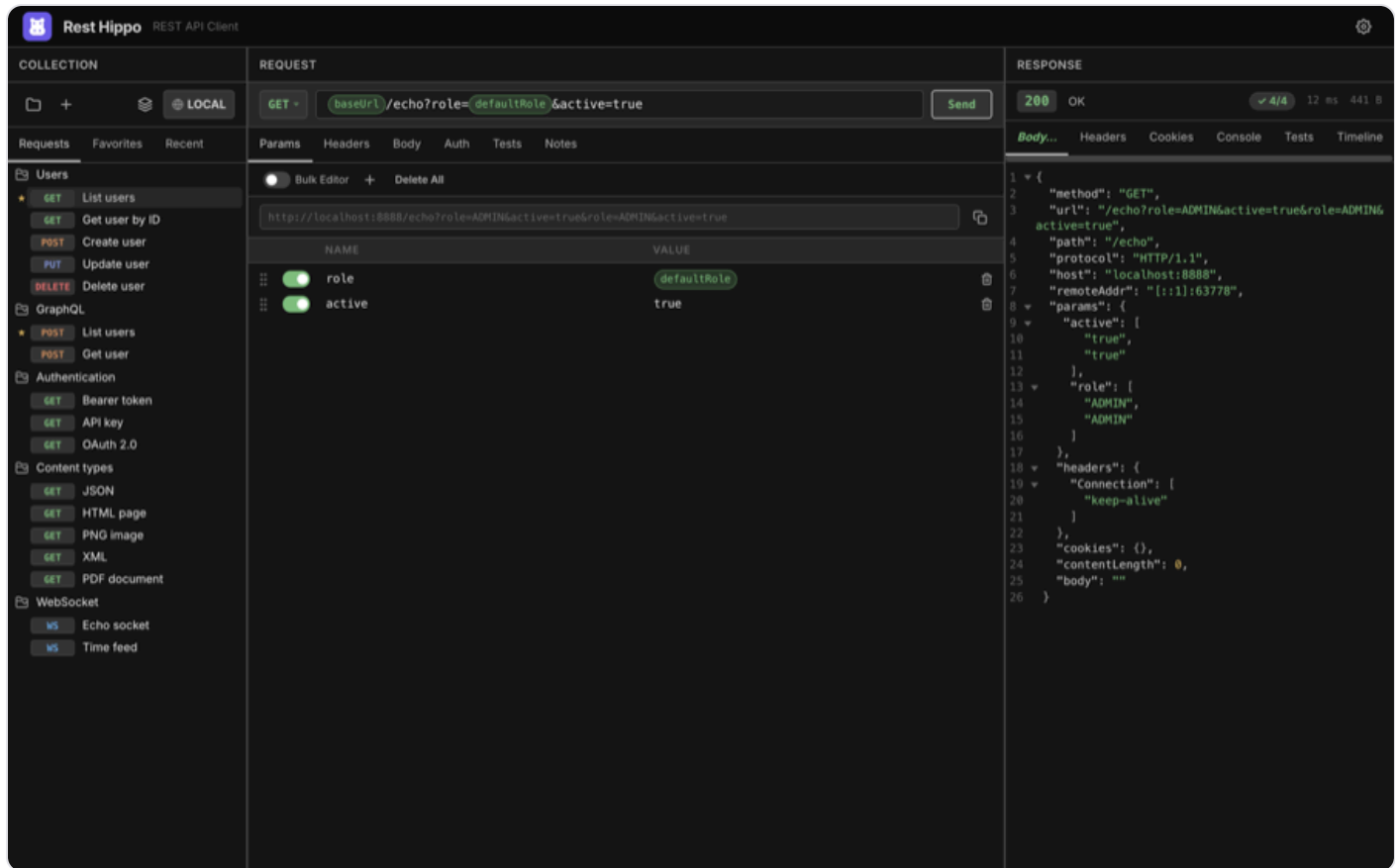
Rest Hippo can keep itself current: when you turn on automatic checks it looks for new releases shortly after launch, downloads them in the background, and asks before restarting to install. Automatic checking is **off by default** — enable it in **Settings → About**, or check any time via **Help → Check for Updates....** See [Settings → About & updates](#).

Rest Hippo can install a `hippo` command so you can start it from a terminal — set this up from [Settings → Command Line](#).

Building from source? See the project [README](#) — `make install` then `make debug` runs Rest Hippo with hot-reload.

The interface

Rest Hippo is organized into three panels:



1. **Collections** (left) — the tree of saved requests. Switch between the **Requests**, **Favorites**, and **Recent** tabs at the top, and press `Cmd + F` (`Ctrl + F`) while the tree is focused to filter it.
2. **Request** (center) — the method selector, URL bar, and the tabs where you define query parameters, headers, the body, authentication, and captures.
3. **Response** (right) — the response status and timing, plus tabs for the body, headers, cookies, console, and timeline.

The header (top right) holds the **Settings** control:

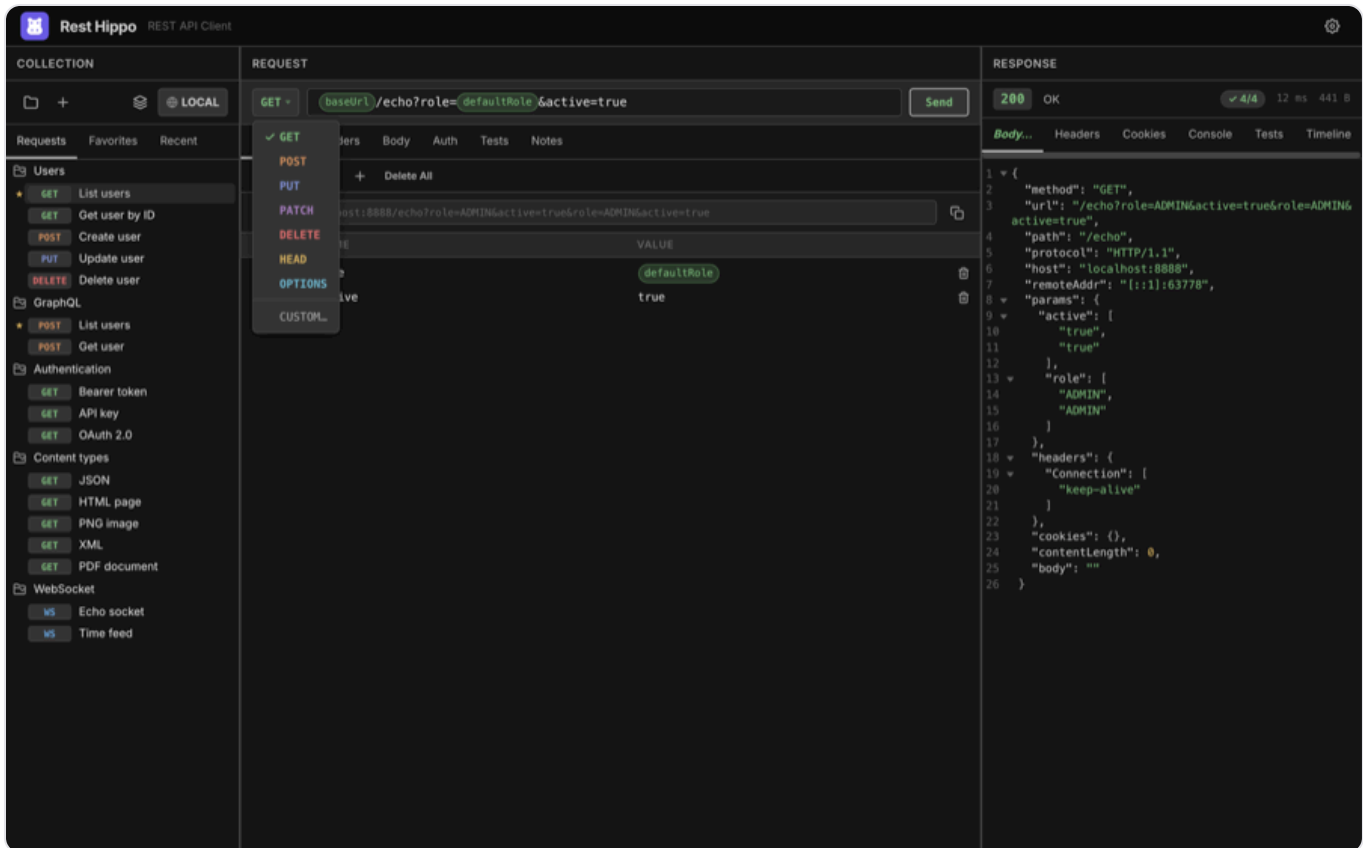
Control	What it does
⚙️ Settings	Opens Settings — theme, fonts, the panel layout , proxy, and more.

The 🌐 **environment selector** (showing the active environment, e.g. `LOCAL`) lives in the **Collections** panel's toolbar next to the collection selector — click it to switch environments, or **right-click** to open the [environments editor](#).

You can resize the panels by dragging the dividers between them, and Rest Hippo remembers the positions.

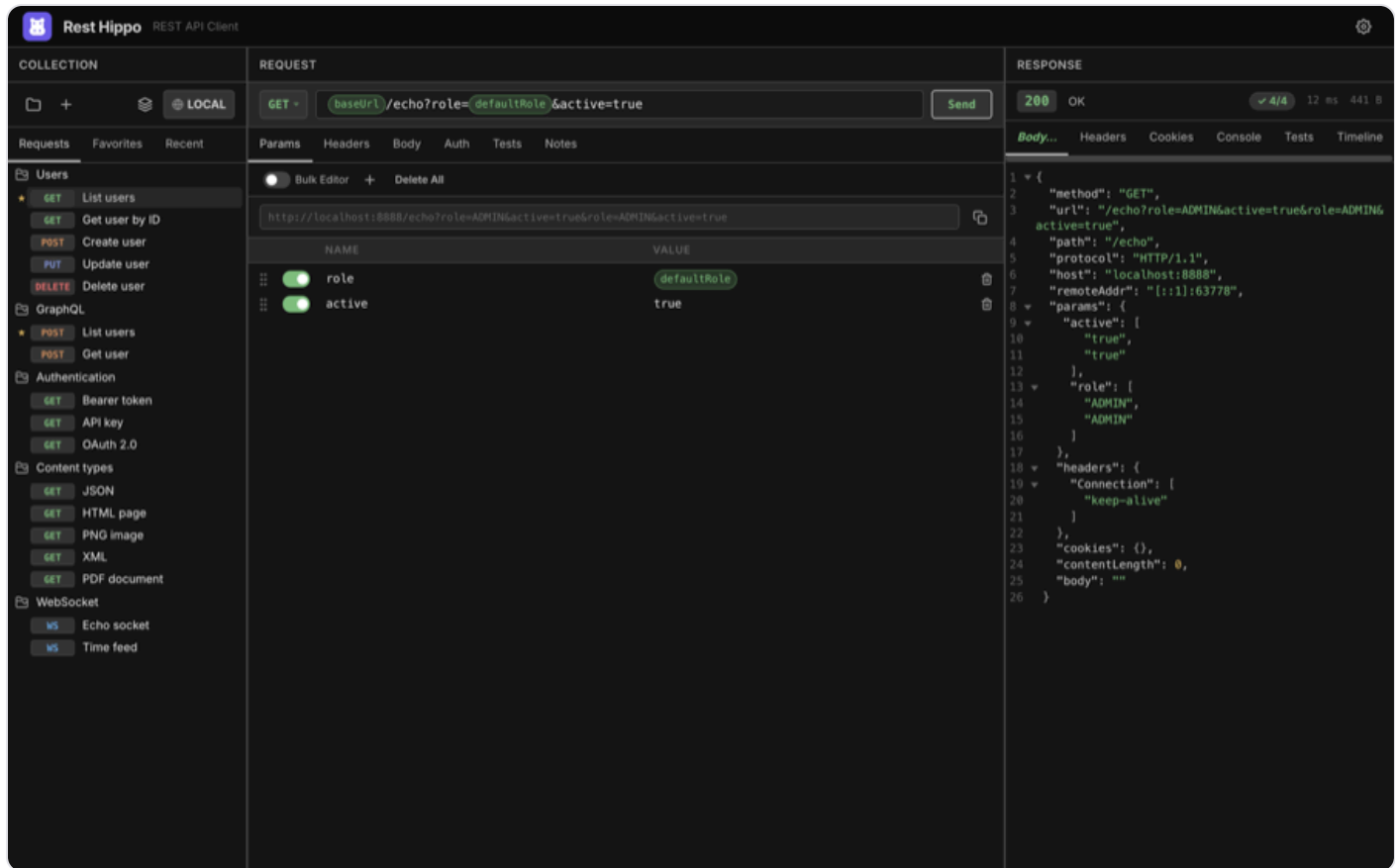
Sending your first request

1. **Pick (or create) a request.** Click a request in the tree to load it. To make a new one, click the **+** (New Request) button above the tree, or right-click a folder and choose **Add Request**.
2. **Choose a method.** Click the method button on the left of the URL bar (`GET` , `POST` , ...) and pick from the menu.



3. **Enter the URL.** Type into the URL bar. You can use `{{variables}}` anywhere — Rest Hippo shows the resolved URL beneath the bar when **Show URL preview** is on (Settings → Appearance).
4. **Add query params, headers, or a body** in the tabs below (all optional). See [Building Requests](#).
5. **Click Send** (or press `Enter` while the URL bar is focused).

The response appears on the right: the status code and text, the elapsed time, the response size, and the body — pretty-printed and syntax-highlighted by default.



Rest Hippo runs requests **natively**, not through a browser, so you're never blocked by CORS. Requests can reach `localhost`, private networks, and any scheme the OS allows.

Diagnostics & logs

Rest Hippo keeps a rotating log of its own activity and errors in a `logs` folder inside your data directory. It records lifecycle and error events — never your secret values — and is your starting point if something goes wrong.

From the **Help** menu:

Item	What it does
Reveal Logs	Opens the log folder in your file manager.
Export Diagnostics...	Saves a single <code>.txt</code> file containing Rest Hippo's version and build info plus the recent logs — ideal to attach to a bug report.

If Rest Hippo ever hits an unexpected error it can't recover from, it writes the details to the log and shows a dialog before closing, so the failure is never silent.

One window at a time. Rest Hippo runs as a single instance to protect your data. Launching a second copy simply brings the existing window to the front instead of opening a duplicate.

Supporting Rest Hippo

Rest Hippo is free and always will be — there's no paid tier, license key, trial, or feature locked behind a payment. Every capability is available to everyone.

If the app saves you time and you'd like to say thank you, **Help → Support Rest Hippo...** (also linked from the About window) opens a donation page in your browser with a suggested \$5 tip. It's entirely optional: a donation unlocks nothing, nothing is ever gated or nagged behind it, and the app never tracks or verifies whether you've given. Skip it with zero downside — the gesture is appreciated, never expected.

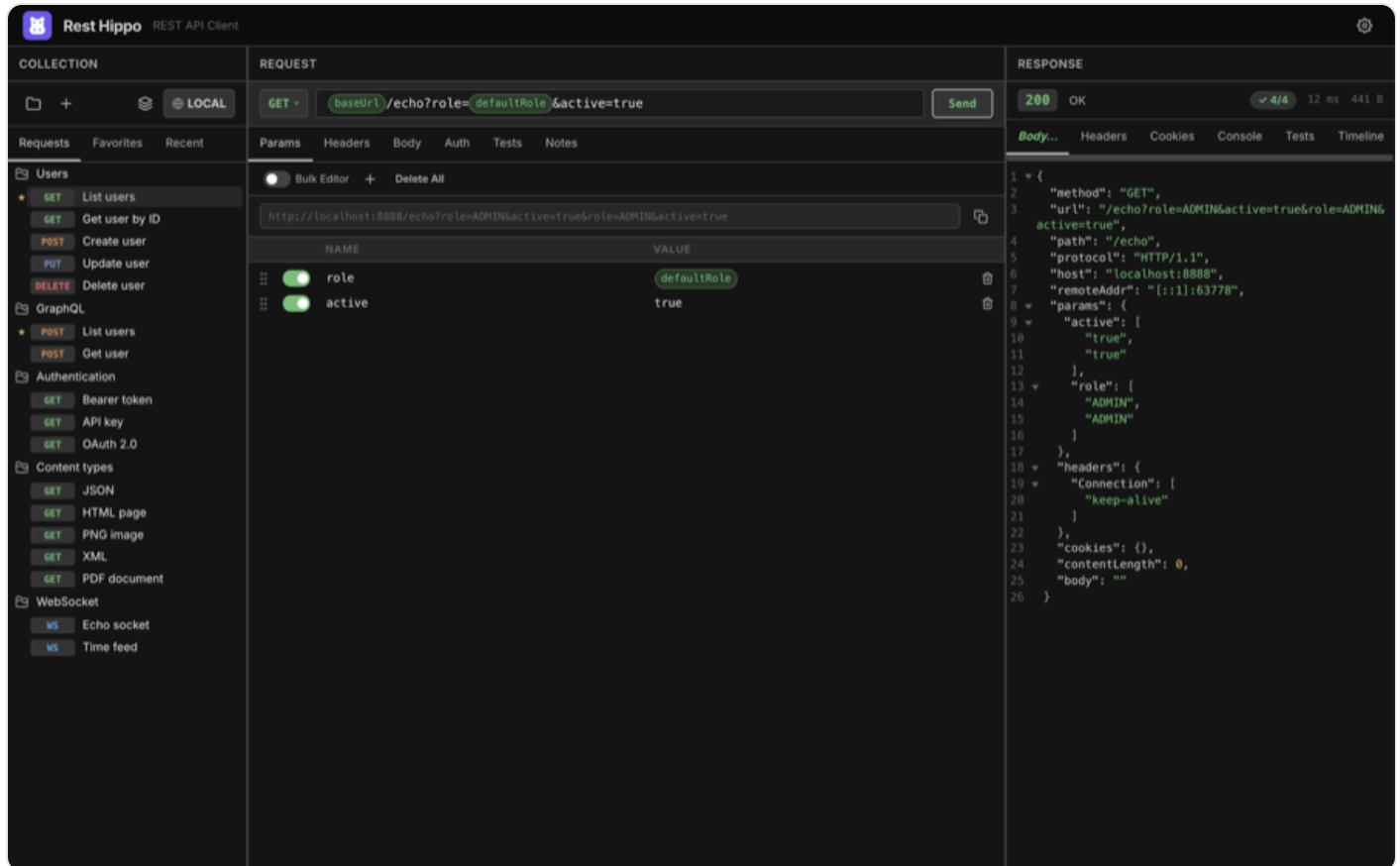
Where to go next

- Organize your work into [collections and folders](#).
- Reuse values with [variables and environments](#).
- Secure your requests with [authentication](#).
- Dig into the [response viewer](#).

Collections & the Tree

[← Back to contents](#)

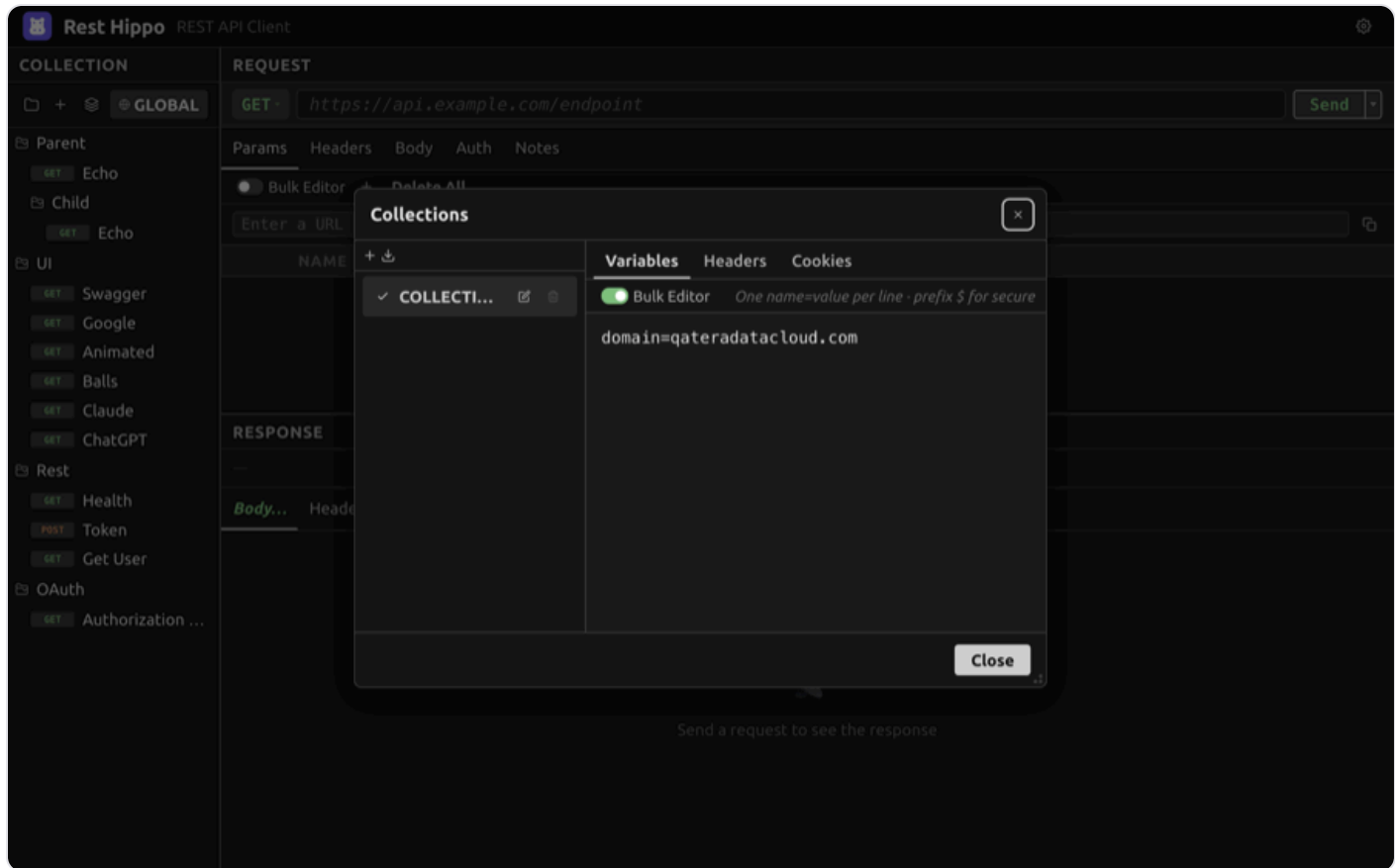
The left panel is where your requests live. Rest Hippo organizes them into **collections** (top-level groups) containing **folders** and **requests**, nested as deeply as you like.



Collections

A **collection** is a self-contained group of requests with its own [environments](#) and an optional [cookie jar](#). The active collection is shown in the **collection selector** in the panel's toolbar (the row with the + buttons), on the right — a stacked-layers icon plus its name (e.g. **Demo API**) — sitting next to the [environment picker](#).

Click the collection selector to open the **Collections** manager, where you create, rename, switch, and delete collections:



- **+** **—** create a new, empty collection.
- Click a collection row to **make it active**; a check marks the active one.
- The pencil and trash icons (or a double-click on the name) **rename** and **delete** a collection. Deletes ask for confirmation.
- The **Environments**, **Headers**, and **Cookies** tabs on the right manage that collection's environments (Global + named), default headers, and stored cookies.

Default headers

The **Headers** tab in the Collections manager sets headers that apply to **every request in the collection** — handy for a shared `Accept`, `X-API-Version`, or a base `Authorization`. It is the same editor as a request's **Headers** tab: a list / bulk-text toggle, drag to reorder, and the standard-header name/value suggestions.

These defaults are merged into each request automatically — at **Send**, **Copy as cURL**, and **Generate code** time:

- A request header with the **same name** (case-insensitive) **overrides** the collection default for that request.

- A **disabled** request header of that name **suppresses** the collection default for that request (a per-request opt-out).
- Authentication headers (e.g. a Bearer token) are still applied last and win over a collection default `Authorization`.

`{{variable}}` tokens work in header values, resolved with the same [variable scopes](#) as a request.

Folders and requests

Inside a collection you build a tree of folders and requests:

- **New Request** — the **+** button above the tree, or right-click a folder → **Add Request**.
- **New Folder** — right-click a collection or folder → **Add Folder**.
- **New WebSocket Request** — right-click a folder → **Add WebSocket Request**, or right-click the **+** button above the tree and choose **Add WebSocket Request** (see [WebSockets](#)).
- **Reorder / re-nest** — hover a request or folder to reveal a drag grip on its right edge, then drag the grip to move it; a placeholder shows where it will land.
- **Rename** — double-click the name (or right-click → **Rename**). `Enter` confirms, `Esc` cancels.
- **Filter** — click anywhere in the tree and press `Cmd + F` (`Ctrl + F` on Windows/Linux) to reveal a filter box above the list; type to filter requests by name. `Esc` hides it and clears the filter.

Each request shows a colored **method badge** (GET, POST, PUT, DELETE, ...). You can switch these badges to compact icons in [Settings → Appearance](#).

The right-click menu

Right-clicking a request or folder opens a context menu with the most common actions:

Collections and folders are the same kind of container, so they share one menu; a request has its own. (A collection's icon differs, but its actions match a folder's.)

Action	Applies to	What it does
Add Request / Add WebSocket Request / Add Folder	collections & folders	Create a child item (also on a request, to insert a sibling after)
Run All Requests	collections & folders	Run every request inside and tally their tests
Rename	any item	Edit the name inline
Favorite / Unfavorite	requests	Toggle the Favorites star
Duplicate	any item	Copy the item (and its contents)
Generate code...	requests	Preview the request as cURL, JavaScript <code>fetch</code> , Python <code>requests</code> , Go, or HTTPie, then copy
Copy as cURL	requests	Copy an equivalent <code>curl</code> command straight to the clipboard
Export...	collections & folders	Export the collection or folder subtree
Clear Run History	requests	Discard the request's saved timeline
Delete	any item	Remove the item (asks to confirm)

To edit a container's [variables](#), **click** the collection or folder in the tree — its variable editor opens in the center panel, replacing the request editor.

Running a whole folder

Right-click a collection or folder and choose **Run All Requests** to send every request inside it — including those in nested sub-folders — one after another. WebSocket requests are skipped (they hold an open connection rather than running once). The entry is greyed out when the folder has nothing to run.

As the run proceeds, a small **pass/total badge** appears on the folder row and counts up live — for example `7/10` means seven of ten [tests](#) have passed so far. Both the no-code [Tests](#) assertions and scripted `hippo.test()` calls count toward the total. When the run finishes the badge settles to its final tally and turns **green** if every test passed, or **red** if any test failed (or a request could not be sent). Hover the badge for a summary, and a toast reports the result.

Nested folders roll up. Every folder in the run gets its own badge: each sub-folder shows the tally for just its own requests, while the folder you clicked rolls up its immediate requests **plus** everything in its

sub-folders. So running a folder of three requests that also contains a sub-folder of two requests shows **2/2** on the sub-folder and **5/5** on the parent.

Clearing the counts. Once a folder shows a badge, its right-click menu gains a **Clear Test Counts** entry just below **Run All Requests** (it is hidden when there is no badge). Choosing it removes the badge from that folder **and every sub-folder beneath it**. This only resets the on-screen counts — each request's recorded **timeline** is left intact.

Requests run **in order**, sharing the collection's **cookie jar**, and any **variables they capture** or scripts write are applied before the next request runs — so a folder that logs in first and reuses the captured token in later requests works as expected.

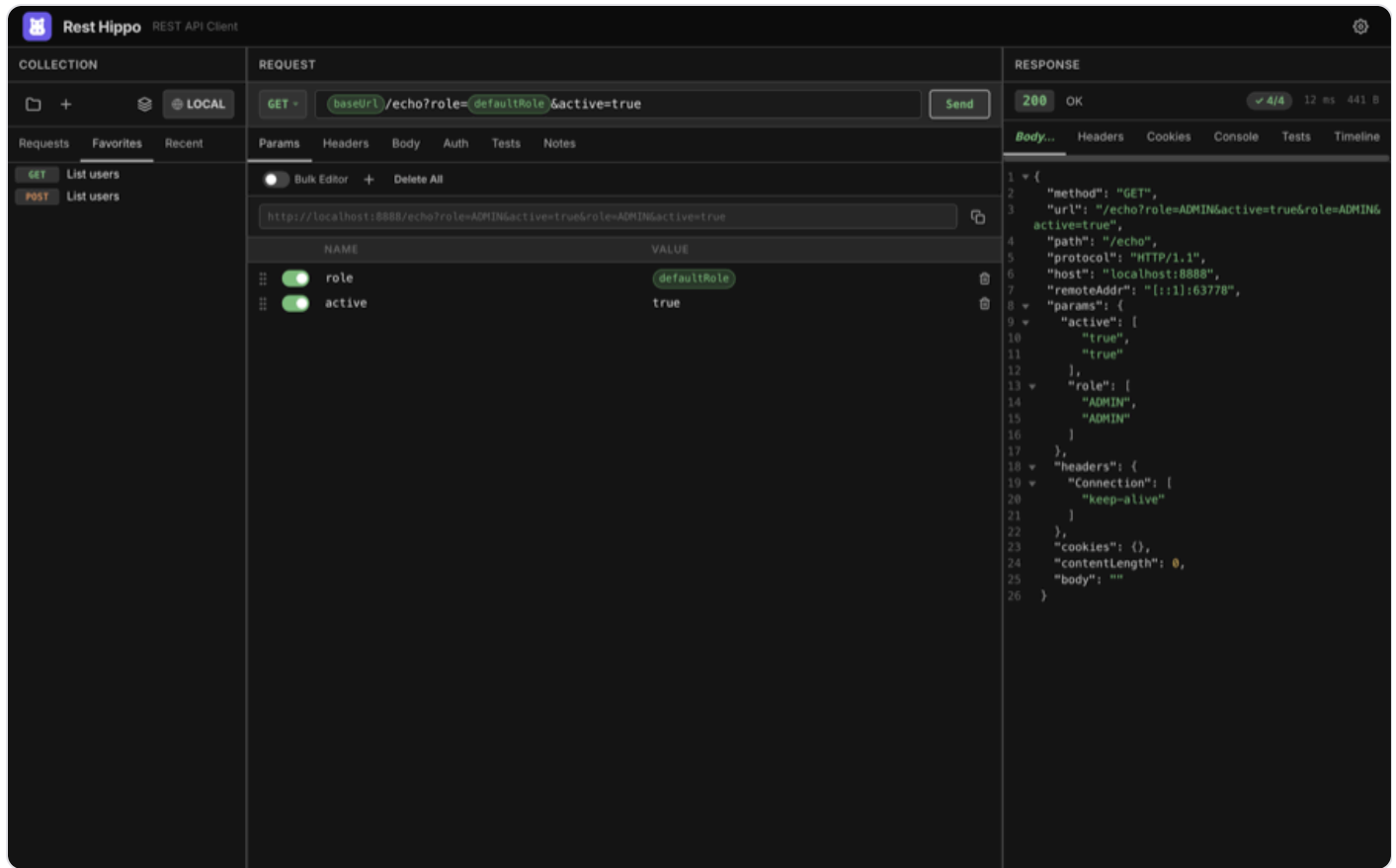
Each request is treated **exactly as if you had opened and sent it yourself**: the run is saved to that request's **timeline** (with its test results), and selecting the request afterwards shows its latest response, console, and Tests tab. The response viewer is not switched while the folder runs — open any request when it finishes to inspect its recorded run.

Folders whose requests use **OAuth 2.0** are the one exception: that flow needs the interactive token popup, so those requests are sent without it during a folder run. Send them individually from the editor when you need a fresh OAuth token. The badge is a live, in-session indicator — it clears when you restart Rest Hippo or run the folder again.

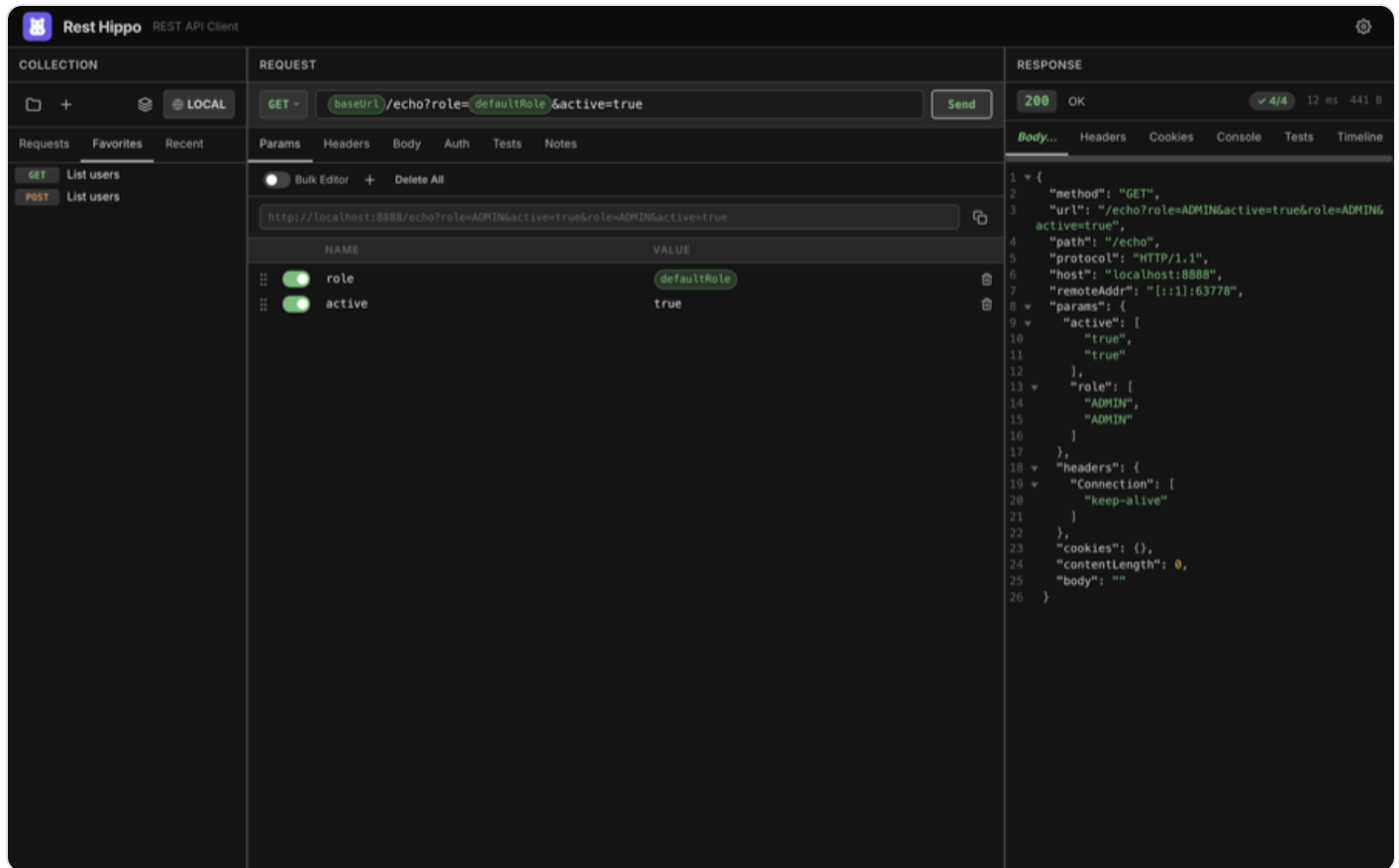
Favorites and Recent

Two extra tabs sit above the tree and span **all** your collections:

Favorites — requests you've starred for quick access. Star a request from its right-click menu; they appear in the order you starred them. Remove one with **Unfavorite** from its right-click menu.



Recent — the requests you've used most recently, newest first. This list is maintained automatically.



Prefer not to see the Recent tab? Turn off **Show recently used tab** in [Settings](#) → [Appearance](#).

Cookies

Each collection has its own **cookie jar**. When **Send cookies** is enabled for a collection, cookies returned by responses are stored and automatically attached to matching later requests in that collection.

Manage the jar from the **Cookies** tab in the Collections manager, and inspect cookies a response set on the [Cookies](#) tab of the response viewer.

Next: [Building Requests](#) →

Building Requests

[← Back to contents](#)

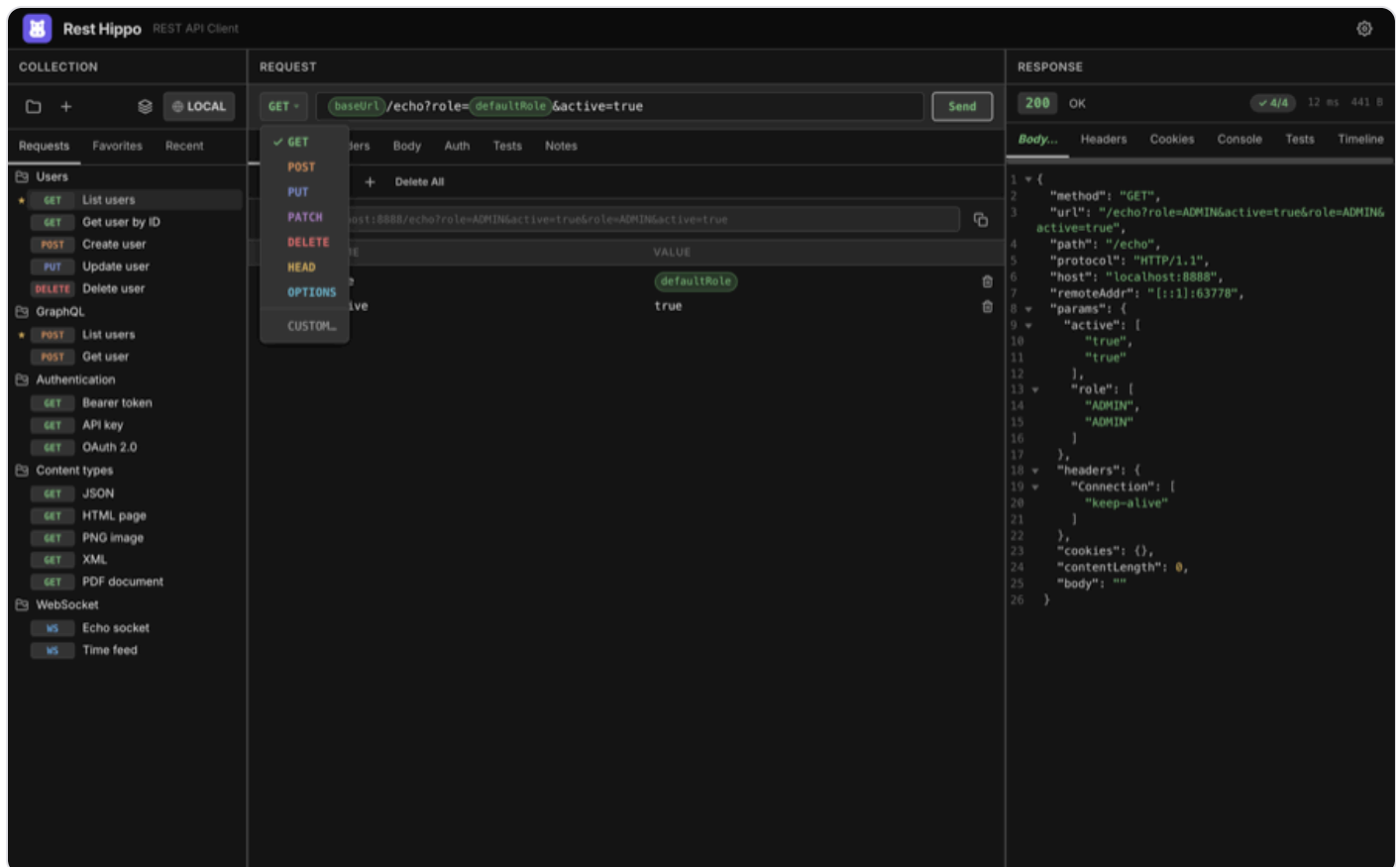
The center panel is the request editor. At the top is the **request bar** — the method, the URL, and the **Send** button. Below it, a row of tabs lets you add query parameters, headers, a body, authentication, and post-response captures.

Edits save automatically. There is no Save button or shortcut — every change you make is written to disk a moment after you stop typing, so your work is always persisted and there's no unsaved state to lose.

The request bar

Method

Click the method button on the left to choose the HTTP verb:



Rest Hippo supports `GET` , `POST` , `PUT` , `PATCH` , `DELETE` , `HEAD` , `OPTIONS` , and a **Custom...** option for any non-standard verb your API expects.

URL

Type the request URL into the bar. You can drop `{{variables}}` anywhere in it — for example `{{baseUrl}}/users/{{userId}}` . When **Show URL preview** is on (Settings → Appearance), Rest Hippo shows the fully-resolved URL directly beneath the URL bar, so you can confirm exactly what will be sent.

Press `Enter` in the URL bar to send the request, or click **Send**. While a request is in flight the button becomes **Stop** — click it to abort.

Send type

An icon to the right of the word **Send** shows how the button fires. Click the caret beside it to open a menu of the three types (the active one is check-marked); the choice is a global default that applies to every request:

Type	Icon	Behavior
Immediate	<i>(none)</i>	Fires the request as soon as you trigger Send (the default).
Delayed	stopwatch	Waits the Delay you set, then fires once.
Interval	recycle arrows	Waits the Interval , fires, then waits the interval again each time a response lands.

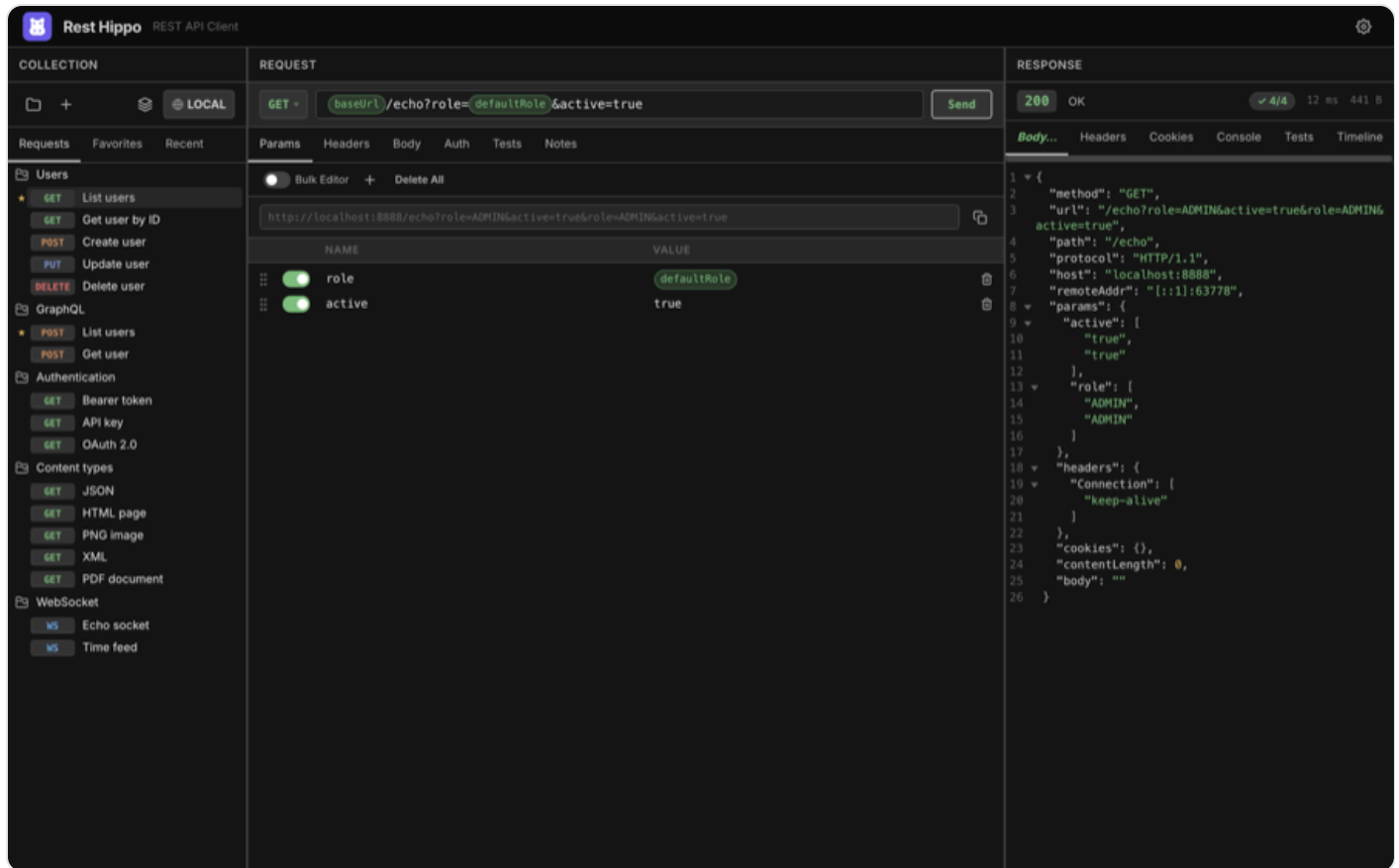
Choosing **Delayed** or **Interval** opens a small dialog under the Send button with a single field for that type's timing — a delay for Delayed, an interval for Interval. Defaults are a **5-second delay** and a **10-second interval**. To adjust the timing later, **right-click the Send button** while it's in Delayed or Interval mode to reopen that same dialog — available only while no countdown is running.

Every trigger — clicking **Send**, pressing `⌘/Ctrl + Enter` , or double-clicking the request in the tree — runs the active type.

While a delayed or interval countdown is ticking, the button reads **Cancel** and a colour sweep drains across it as the timer runs down; click it to cancel the timer and return to **Send**. Switching to another request also stops the countdown.

Query parameters

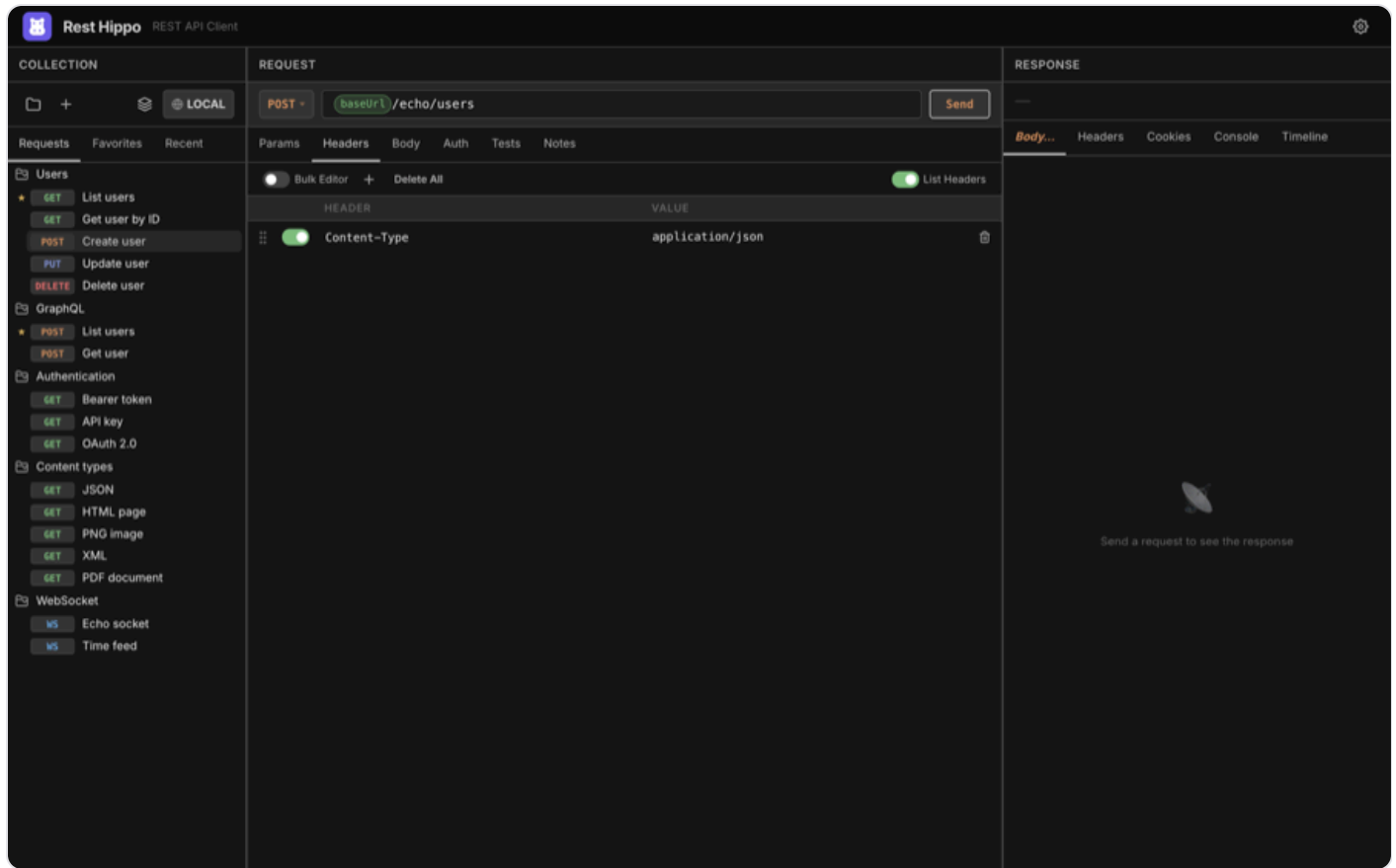
The **Params** tab edits the query string as an editable key/value grid. Rest Hippo keeps it in sync with the URL — editing one updates the other.



- Each row has an **enabled** toggle, a **name**, and a **value**. Disabled rows are kept but not sent.
- Both name and value accept `{{variables}}` .
- Use **+** to add a row, the trash icon to remove one, and **Delete All** to clear them.
- Toggle **Bulk Editor** to edit all parameters as plain text instead of rows — handy for pasting.

Headers

The **Headers** tab works the same way — an enabled/name/value grid with a bulk text mode.

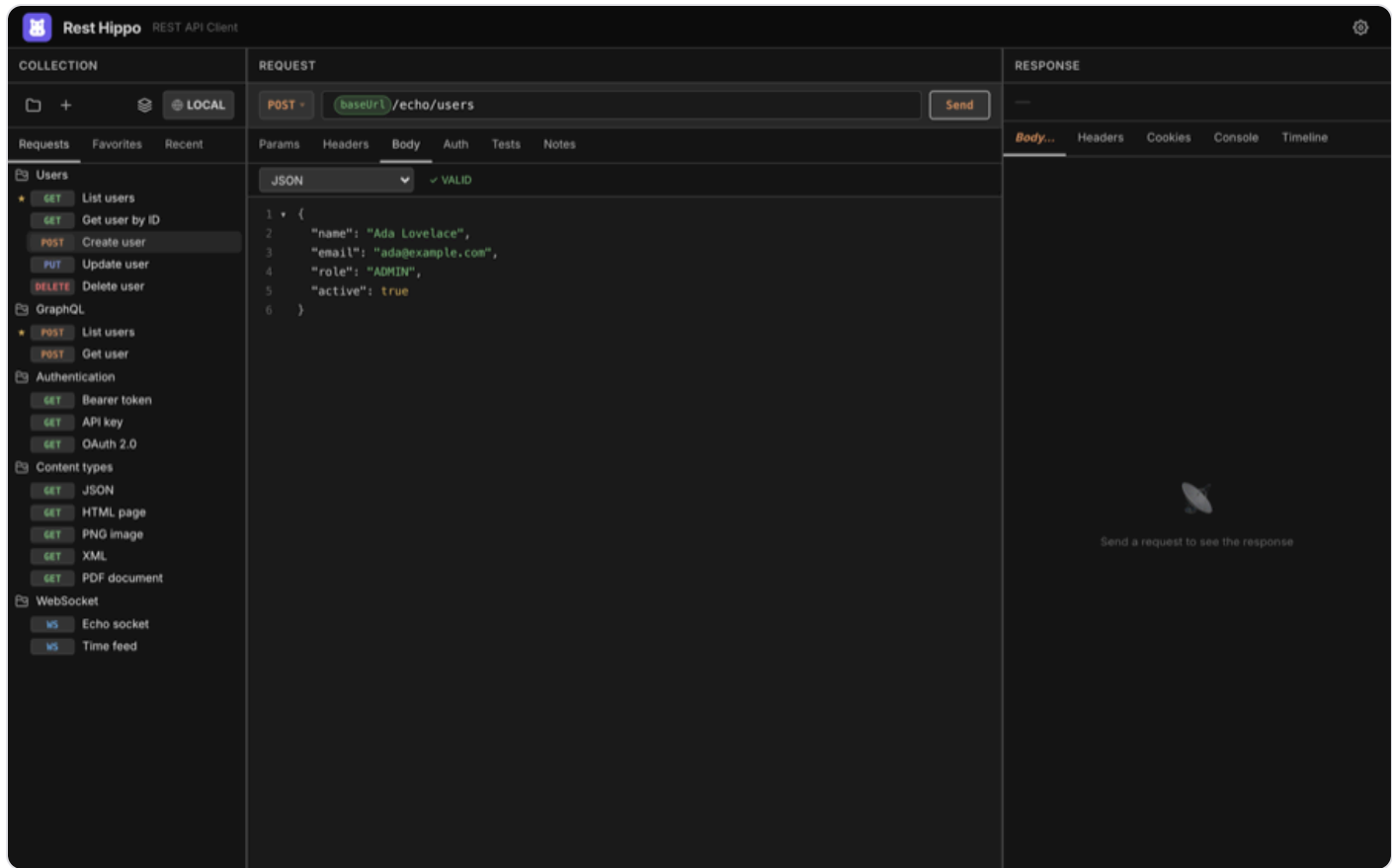


As you type a header name, Rest Hippo suggests standard header names (`Content-Type` , `Authorization` , `Accept` , ...). Values accept `{{variables}}` .

Some headers are managed for you. When you choose an `authentication` type, the matching `Authorization` (or custom) header is added automatically at send time.

Request body

The **Body** tab lets you choose a body type from the dropdown and edit it:



Body type	Use it for
No Body	GET / HEAD and other bodyless requests
JSON	application/json payloads, with syntax highlighting and validation
YAML	YAML payloads
XML	XML payloads
Plain Text	Any raw text
Form Data	multipart/form-data — key/value fields, each Text or File
Form URL Encoded	application/x-www-form-urlencoded key/value pairs
GraphQL	A GraphQL query + variables
File	Send a file's raw bytes as the body

For the structured editors (JSON / YAML / XML), Rest Hippo shows a **✓ VALID** / **✗ INVALID** badge as you type. When the body is invalid, the badge spells out the reason — the line and column plus the parser's message (e.g. `3:12 Unexpected token`) — so you know exactly what to fix; the full text is also

on the badge's tooltip. The code editor has line numbers and a resize handle, and its right-click menu offers **Prettify** and **code folding**. `{{variables}}` are highlighted inline and resolved at send time.

The **Form Data** and **Form URL Encoded** editors use the same key/value grid as Params and Headers, with a bulk-text mode. In Form Data, switch a row between **Text** and **File** to attach a file.

Authentication & captures

Two more tabs round out the request:

- **Auth** — attach credentials (Bearer, OAuth 2.0, ...).
- **Captures** — pull values out of the response into variables for later requests.

There's also a **Notes** tab for free-form Markdown notes attached to the request.

Tests

The **Tests** tab attaches **assertions** that validate the response after every send — a no-code way to turn a request into an API smoke test. (Enable it under **Settings** → **Request** → **Show Tests tab**; it's off by default.)

The screenshot displays the Rest Hippo REST API Client interface. The left sidebar shows a collection of requests under 'Users', 'GraphQL', 'Authentication', 'Content types', and 'WebSocket'. The main panel is divided into three sections: 'REQUEST', 'RESPONSE', and 'TESTS'.

REQUEST section:

- Method: GET
- URL: `baseUrl/echo?role=defaultRole&active=true`
- Buttons: Send
- Tabs: Params, Headers, Body, Auth, Tests, Notes

TESTS section:

SOURCE	HEADER / PATH	MATCHER	EXPECTED
Status	-	equals	200
Response time	-	is less than	1000
Header	Content-Type	contains	json
JSON body	\$.method	equals	GET

RESPONSE section:

- Status: 200 OK
- Body:

```
{
  "method": "GET",
  "url": "/echo?role=ADMIN&active=true&role=ADMIN&active=true",
  "path": "/echo",
  "protocol": "HTTP/1.1",
  "host": "localhost:8888",
  "remoteAddr": "::1:63778",
  "params": {
    "active": [
      "true",
      "true"
    ],
    "role": [
      "ADMIN",
      "ADMIN"
    ]
  },
  "headers": {
    "Connection": [
      "keep-alive"
    ]
  },
  "cookies": {},
  "contentLength": 0,
  "body": ""
}
```

Each row is one check:

- **Source** — what to look at: the **Status** code, the **Response time** (ms), a **Header**, the raw **Body**, or a **JSON body** value (give a path like `$.data.id` in the **Header / Path** column).
- **Matcher** — how to compare: **equals**, **does not equal**, **contains**, **does not contain**, **exists**, **does not exist**, **is less than**, **is greater than**, or **matches** (regular expression).
- **Expected** — the value to compare against (ignored for exists / does not exist).

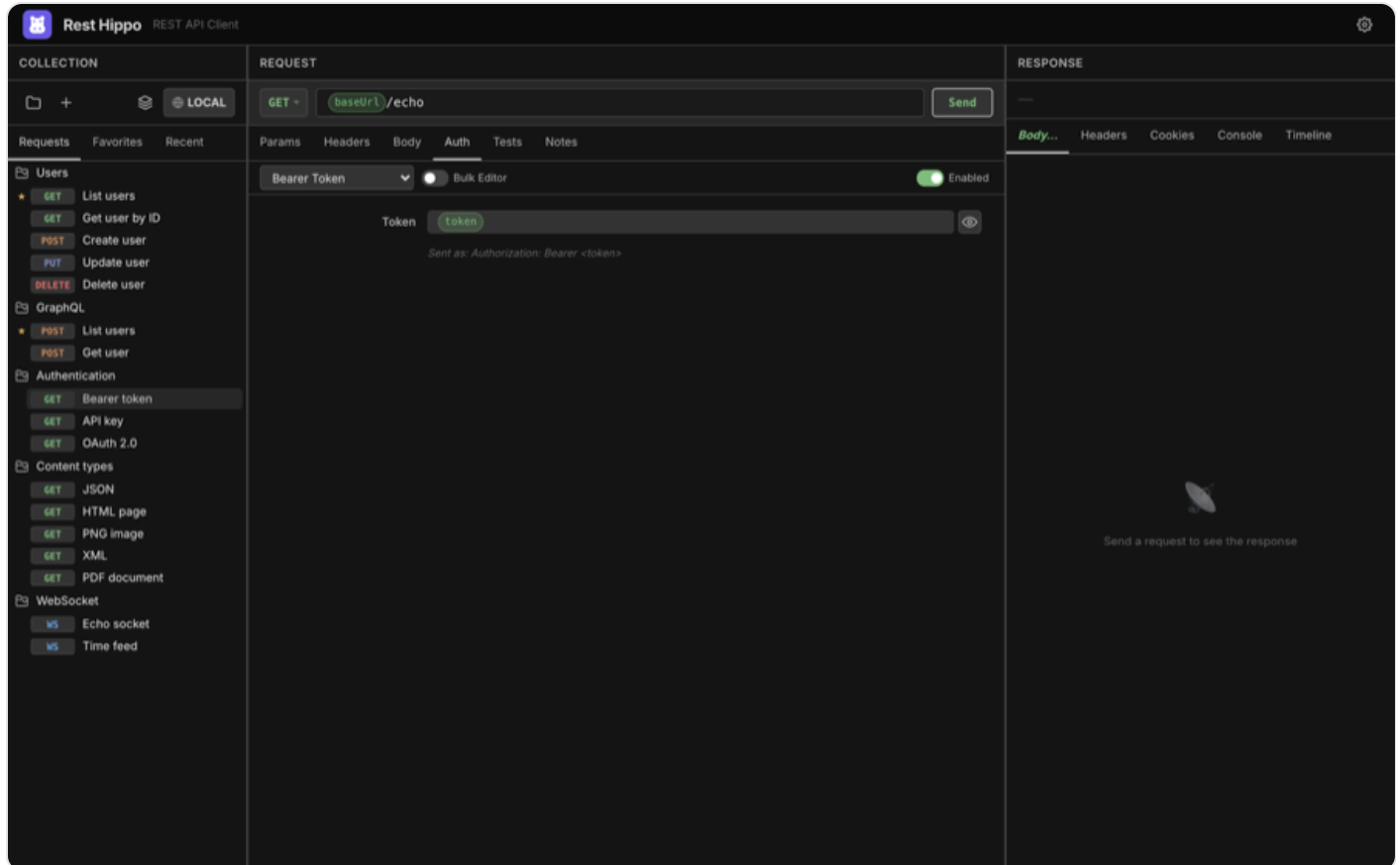
Rows have the familiar enabled toggle, drag-to-reorder, and delete controls. Results show on the response **Tests** tab with a pass/fail badge, and are saved with each run so the **Timeline** shows whether a past run passed. For checks that need logic, write `hippo.test()` **assertions** on the Scripts tab — they run alongside the grid and share the same Tests tab.

Next: [Authentication](#) →

Authentication

[← Back to contents](#)

The **Auth** tab attaches credentials to a request. Pick a type from the dropdown and fill in the fields — Rest Hippo applies the right header (or query parameter) automatically when you send. Use the **Enabled** toggle to turn auth on or off without losing the settings.



Every secret field (passwords, tokens, client secrets, keys) is **masked** with a reveal toggle and is **encrypted at rest**, so credentials never sit in plain text on disk. All fields accept `{{variables}}`, so you can keep secrets in a secure environment variable and reference them here.

Types at a glance

Type	What it sends
None	No credentials
API Key	A key as a header or query parameter
Basic	<code>Authorization: Basic ...</code> (username:password, base64-encoded)
Bearer Token	<code>Authorization: Bearer <token></code>
Digest	RFC 2617/7616 challenge-response (handles the 401 round-trip)
NTLM	Microsoft NTLM handshake (username, password, domain, workstation)
AWS IAM	AWS Signature V4 signing (access key, secret, region, service)
OAuth 1.0	OAuth 1.0a request signing (<code>Authorization: OAuth ...</code>)
OAuth 2.0	A full OAuth 2.0 / OIDC flow (see below)

API Key

Enter a **key name** and **value**, and choose whether to add it to the **Header** or the **Query Param**. Rest Hippo suggests common key names as you type.

Basic

Enter a **username** and **password**. Rest Hippo base64-encodes them into the `Authorization` header.

Bearer Token

Enter a **token**. Rest Hippo sends `Authorization: Bearer <token>`. The tip line under the field shows exactly what will be sent.

Digest & NTLM

For **Digest**, supply a username and password; Rest Hippo performs the challenge-response handshake (including the initial 401) for you. **NTLM** adds optional **domain** and **workstation** fields for the multi-leg Windows handshake.

AWS IAM (SigV4)

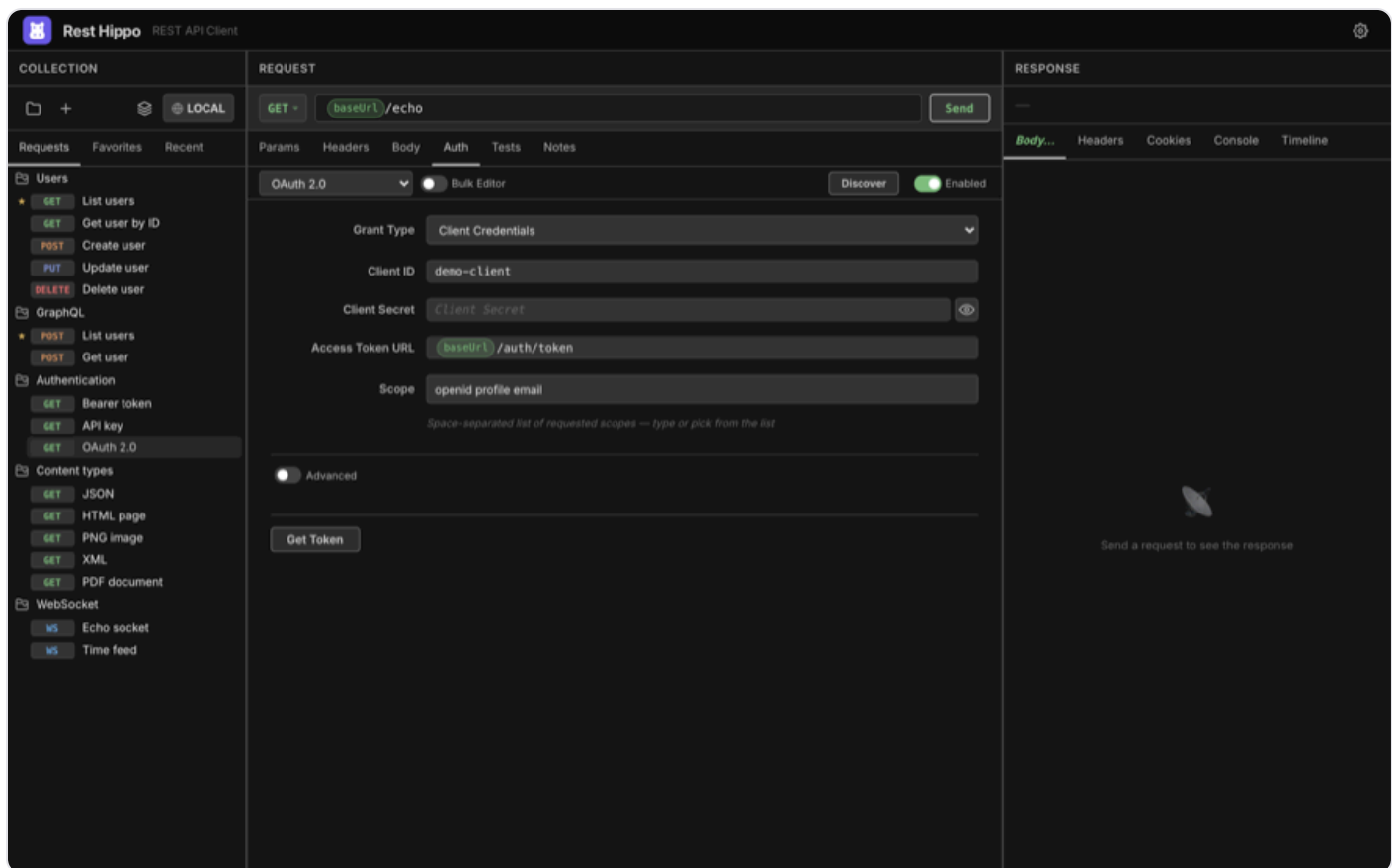
Sign requests with **Signature Version 4**: enter your **Access Key ID**, **Secret Access Key**, **Region**, and **Service** (Rest Hippo can infer the service from the host), plus an optional **Session Token**. The signature, date, and signed-headers are computed at send time.

OAuth 1.0a

For legacy APIs that still use **OAuth 1.0a**, enter your **Consumer Key** and **Consumer Secret**, and (for an access token) the **Token** and **Token Secret**. Choose a **Signature Method** — **HMAC-SHA1** (the most common), **HMAC-SHA256**, or **PLAINTEXT** — and optionally a **Realm**. Rest Hippo signs the request at send time, computing the signature over the method, URL, and parameters and adding the `Authorization: OAuth ...` header; the `oauth_nonce` and `oauth_timestamp` are generated automatically for each request.

OAuth 2.0

The OAuth 2.0 type supports the full set of grant types and OIDC discovery:



1. **Grant Type** — choose Authorization Code, Implicit, Client Credentials, Resource Owner Password, **Device Authorization**, or **Token Exchange**.
2. Fill in **Client ID**, **Client Secret** (for confidential clients), the **Authorization URL** and **Access Token URL**, and the **Scope** (with autocomplete for common scopes like `openid`, `profile`, `email`).
3. Click **Get Token**. For browser-based flows (Authorization Code, Implicit), Rest Hippo opens a popup window for you to log in, intercepts the redirect, and exchanges the code for an access token. The

token is then attached to your request.

Device Authorization (RFC 8628) — for input-constrained or headless clients. Provide the **Device Authorization URL**, **Access Token URL**, and **Client ID**, then click **Get Token**: Rest Hippo shows a **user code** and a **verification URL** to open on another device, and polls in the background until you approve (handling the standard `authorization_pending` / `slow_down` back-off) or the code expires.

Token Exchange (RFC 8693) — swap one token for another. Provide the **Subject Token** and its **Subject Token Type**, and optionally an **Actor Token** (for delegation), **Requested Token Type**, **Audience**, and **Resource**. The exchanged token is cached like any other grant.

OIDC Discovery — click **Discover** to fetch a provider's `.well-known/openid-configuration` and auto-populate the authorization and token URLs and the available scopes.

Advanced — expand the **Advanced** section for fine control over **Response Type**, **State**, where credentials go (`header` vs `body`), **Audience**, **Resource**, **Origin**, and a custom **Header Prefix**.

Authorization Code flows use **PKCE** by default for public clients, so you can authenticate against modern providers without embedding a client secret.

Next: [Variables & Environments](#) →

Variables & Environments

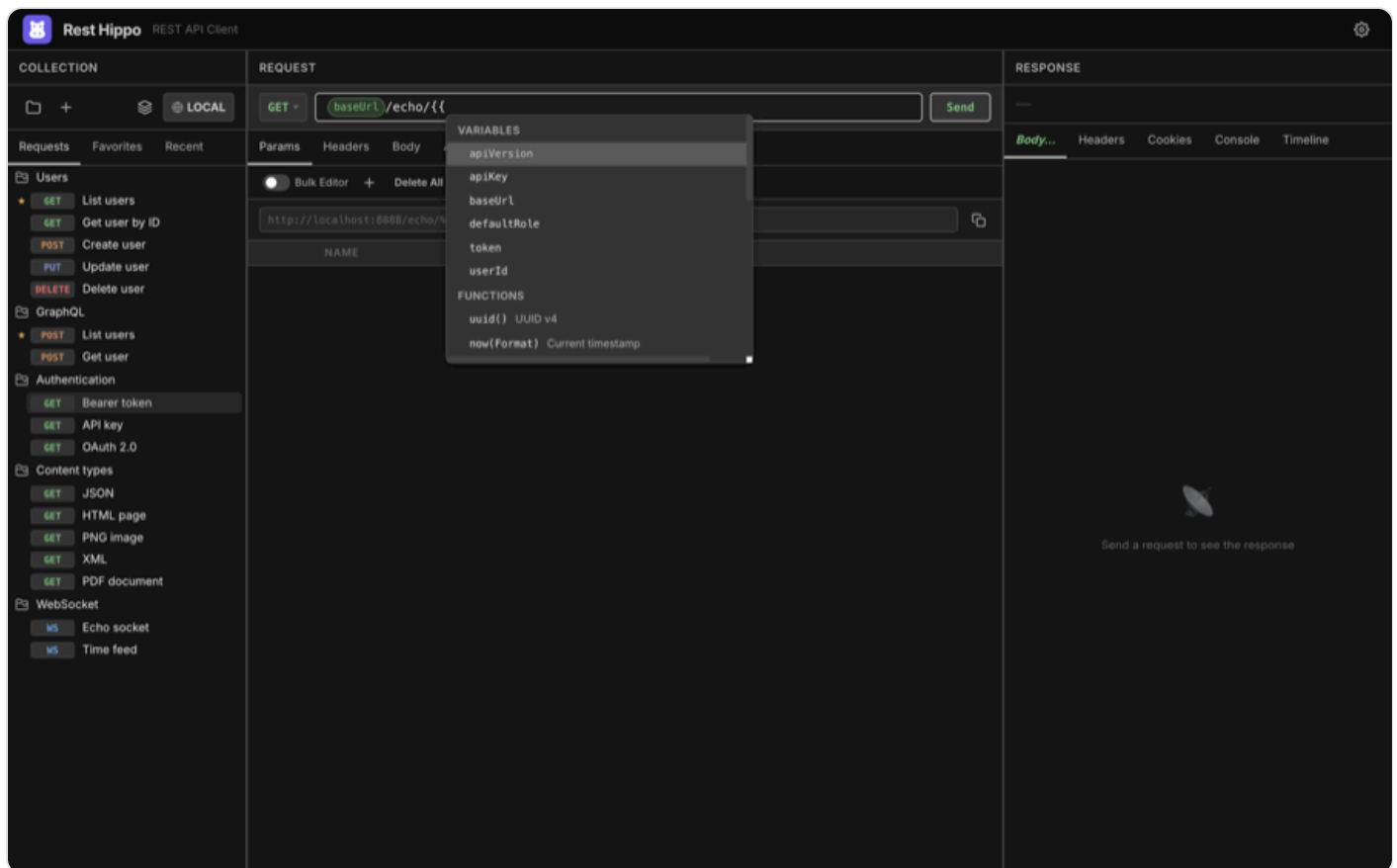
[← Back to contents](#)

Variables let you write a request once and reuse it across hosts, users, and runs. Anywhere you can type — the URL, params, headers, body, or auth fields — you can insert a variable with double-brace syntax:

```
{{name}} .
```

Using variables

Type `{{` in any field to open the **typeahead**, which lists the variables available in the current scope. Pick one and Rest Hippo inserts it as a highlighted **pill**:



At send time, every `{{name}}` is replaced with its resolved value. With [Show URL preview](#) on, you can see the resolved URL before you send.

Scopes

A variable can be defined at three levels. When the same name exists in more than one, the **most specific scope wins**:

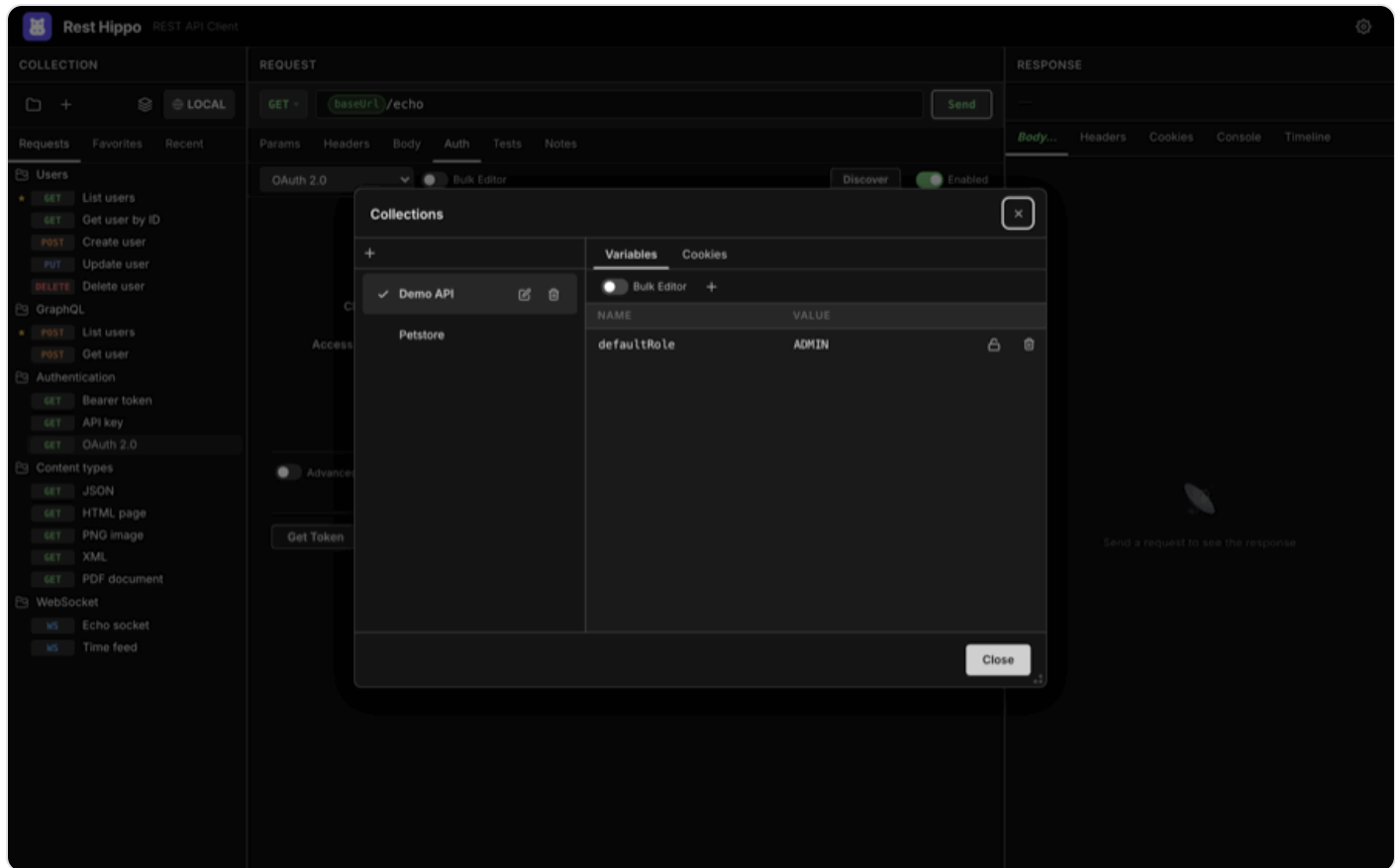
```
Folder ▶ Environment ▶ Global
(most specific)      (most general)
```

- **Global** — available to every request in the **active collection** (each collection has its own Global set — see [Environments](#) below). Global is the collection-wide tier: variables you want every request in the collection to see go here.
- **Environment** — defined in the active environment (see below). Switching environments swaps these values.
- **Folder** — defined on a folder, scoped to the requests inside it.

So a `baseUrl` set in your **Local** environment overrides a global `baseUrl`, and a folder-level override beats them both.

Folder and collection-wide variables

Click a folder in the tree to edit its variables right in the center panel: the request editor is replaced by an inline variable editor scoped to that folder, and selecting a request switches the panel back. It offers the same bulk-text and key/value modes — and per-variable secure masking — as the other variable editors, and saves as you type.



Clicking a collection edits its **Global** environment in that same inline editor — collections no longer have a separate variable set; their collection-wide variables are simply the Global environment (also editable from the [Collections manager](#)'s **Environments** tab).

Environments

An **environment** is a named set of variables you can switch between — the classic use is one environment per deployment (Local, Staging, Production), each with its own `baseUrl`, credentials, and IDs.

Environments belong to the collection. Each collection has its own **Global** set and its own list of named environments, with its own active selection. When you switch the active collection, the environment picker and the active environment switch with it — so a `Staging` environment in one collection is entirely independent of a `Staging` in another. A brand-new collection starts with an empty Global set and no named environments.

Click the **environment picker** in the collections toolbar (it shows the active environment, e.g. `LOCAL`) to open a quick-switch menu. It lists **Global** followed by every named environment, with a check beside the active one — pick one to **make it active** instantly (all `{{variables}}` resolve against it).

To **add, delete, or edit** environments, open the **Environments** tab of the [Collections manager](#) — right-click the environment picker and choose **Manage...**, press **⌘/Ctrl + E**, or open the collections manager and switch to the **Environments** tab. (A plain click on the picker only switches the active environment.)

- The **Environment** dropdown at the top selects which environment you're editing: **Global** (always present) or any named environment. Choosing one also makes it the active environment, kept in sync with the toolbar picker.
- **+** adds a named environment (type its name and press Enter); the **trash** button deletes the selected environment (with a confirmation). Global can't be deleted.
- Below the selector, the variable editor edits the selected environment's variables as a key/value grid (or as plain text via **Bulk editor**). Changes save automatically.

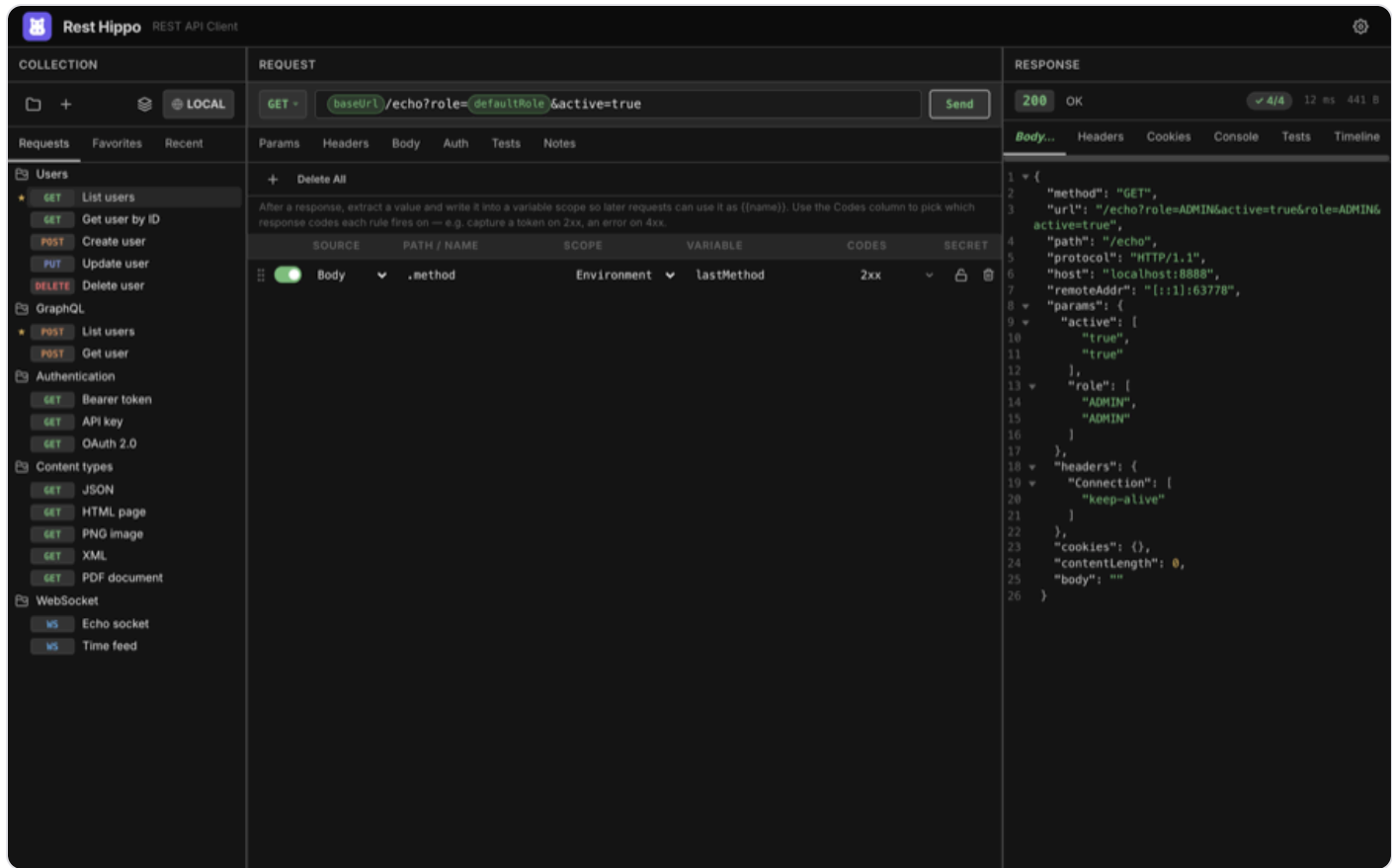
Secure variables

Mark a variable **secure** (the lock icon) to **encrypt it at rest** and mask it in the UI. Revealed secrets re-mask themselves automatically after a short time. Use secure variables for tokens and passwords you reference from [auth fields](#).

Captures

Captures close the loop: after a request succeeds, pull a value **out** of the response and write it into a variable that later requests can use. A login request can capture the returned token; the next request sends it as a Bearer token — no copy-paste.

Configure them on the **Captures** tab of the request:



Each capture rule has:

Field	Meaning
Source	Where to read from: the Body , a Header , or the Status code
Path / Name	For Body, a dot-path like <code>.data.token</code> ; for Header, the header name
Scope	Where to write: Environment or Global
Variable	The variable name to write
Codes	Which response codes the rule fires on (see below)
Secret	Mark the captured value secure (encrypted, masked)

For the **Body** source the same dot-path (`.data.token` , `.items.[0].id`) works whether the response is **JSON**, **YAML**, or **XML** — the body is parsed automatically and the path is walked over it. XML is addressed from its root element (e.g. `<auth><token>...</token></auth>` → `.auth.token`), and repeated tags become an indexable list (`.list.item.[0]`). A body that isn't one of those formats, or a path that finds nothing, is reported as a warning and captures nothing.

Choosing which response codes a rule fires on

Each rule has its own **Codes** selector — click it to open a checklist where you can tick whole status **groups** (`1xx` – `5xx`), choose **Any status**, or type **specific codes** (e.g. `201` , `404`) that appear as removable chips. A rule runs only when the response status matches its selector, so a single request can capture different values into different variables depending on the outcome — for example capture the access token from a `2xx` body into `token` , but capture the error message from a `4xx` body into `lastError` .

New rules default to **2xx**, so captures keep firing only on success unless you opt a rule into other codes. When a value can't be found (missing field, empty body) the rule is reported as a warning and never overwrites a good value with an empty one.

Rest Hippo shows a small toast confirming what was written (e.g. *Captured 1 variable → env.token*), and the response status bar shows a **captured** badge.

Next: [Functions](#) →

Functions

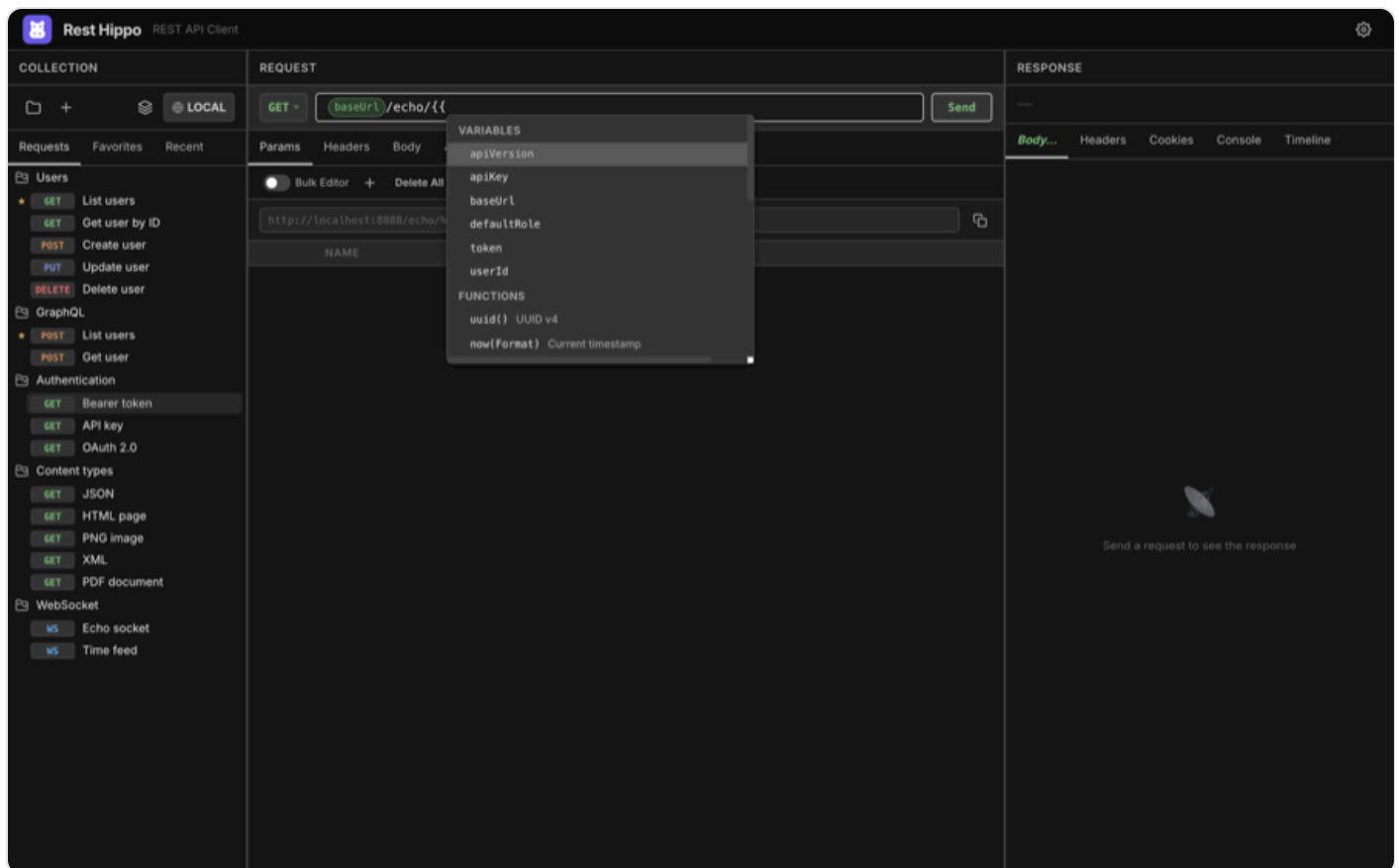
[← Back to contents](#)

Functions are dynamic `{{ }}` tokens that compute a value **at send time** instead of looking one up from a [variable](#). Use them to stamp a request with a fresh UUID, the current timestamp, a Base64-encoded credential, an HMAC signature, or a value pulled from another request's response.

Where a `{{variable}}` resolves to a stored value, a function like `{{uuid()}}` **runs** every time the request is sent — so each send gets a new result.

Inserting a function

Type `{{` in any field to open the **typeahead**. Below your variables it lists the available functions, grouped into sections:



Pick a function and Rest Hippo inserts it as a highlighted **pill**. If the function takes arguments, the **pill editor** opens so you can fill them in; click an existing function pill any time to edit its arguments again.

Under the hood a function pill is just text in the form `{{name("arg1", "arg2")}}` — arguments are positional and double-quoted — so you can also type one by hand. `{{uuid()}}` takes no arguments; `{{now("Unix")}}` takes one.

Built-in functions

General-purpose helpers that resolve instantly, with no network or context needed.

Function	Result
<code>uuid()</code>	A random UUID v4 , e.g. <code>9f1c...-...</code> . A fresh one on every send.
<code>now("ISO")</code>	The current timestamp . Format: <code>ISO</code> (default), <code>Unix</code> (seconds), <code>UnixMs</code> (milliseconds), or <code>RFC2822</code> .
<code>base64encode("value")</code>	Base64-encode the value (UTF-8 safe).
<code>base64decode("value")</code>	Base64-decode the value.
<code>urlEncode("value")</code>	Percent-encode the value for use in a URL.
<code>urlDecode("value")</code>	Percent-decode the value.
<code>randomInt("0", "100")</code>	A random integer between min and max, inclusive (defaults <code>0 – 100</code>).

Arguments can themselves contain variables — e.g. `{{base64encode("{{user}}:{{pass}}")}}` builds a Basic-auth credential from two variables.

Context functions

These read from **where the request lives** and the active environment. They take no network round-trip.

Function	Result
<code>collectionName()</code>	The name of the active collection .
<code>requestName()</code>	The name of the current request .
<code>environmentName()</code>	The name of the active environment .
<code>folderName("0")</code>	An ancestor folder's name. <code>0</code> is the immediate parent, <code>1</code> its parent, and so on.

Request Outputs

Chain requests together: pull a value **out of another request's response** and feed it into this one. Reference the source request by name.

Function	Result
<code>response("Login", ".data.token")</code>	A value from the named request's response body . The second argument is a dot-path (<code>.data.token</code> , <code>.items.[0].id</code>); <code>.</code> returns the whole body.
<code>responseHeader("Login", "Location")</code>	A single response header from the named request.
<code>responseStatus("Login")</code>	The HTTP status code of the named request's last response.
<code>run("Seed data")</code>	Runs the named request before this one. It produces no text of its own (it resolves to an empty string) — use it purely to trigger a prerequisite.

The dot-path is the same syntax as [response captures](#) and works over JSON, YAML, or XML bodies.

Refresh mode

`response` , `responseHeader` , and `responseStatus` each have a **Refresh mode**:

- **Use last result** (default) — read the value from the source request's most recent response, without re-sending it. Fast, but the source must have been run at least once.
- **Run immediately before** — send the source request first, every time, then read from its fresh response. Use this when the value is short-lived (e.g. a one-time token) and must be current on every send.

Backend functions

These run in Rest Hippo's main process (where the crypto and OS APIs live) and resolve asynchronously.

Function	Result
<code>hmac("SHA256", "key", "message")</code>	An HMAC signature. Algorithm: <code>SHA256</code> (default) or <code>SHA512</code> .
<code>hash("SHA256", "value")</code>	A hash of the value. Algorithm: <code>SHA256</code> (default) or <code>SHA512</code> .
<code>environmentVariable("RESTHIPPO_API_KEY")</code>	An OS environment variable . For safety, only names beginning with <code>RESTHIPPO_</code> are readable — any other name returns empty, and the prefix is <i>not</i> stripped.

`environmentVariable` lets you keep a secret out of your stored collection entirely: export `RESTHIPPO_API_KEY=...` in your shell before launching Rest Hippo, then reference it with `{{environmentVariable("RESTHIPPO_API_KEY")}}` .

Next: [GraphQL](#) →

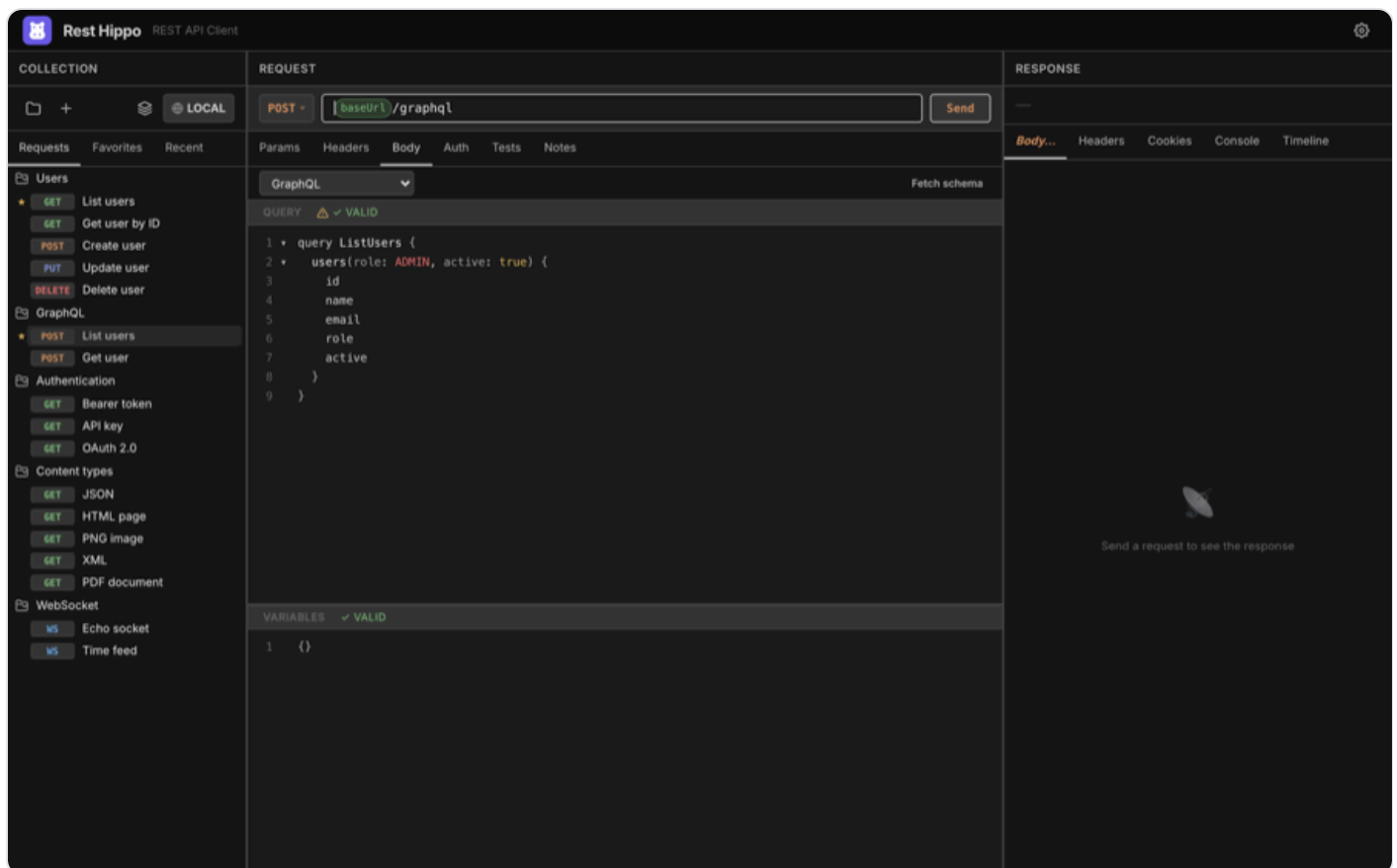
GraphQL

[← Back to contents](#)

Rest Hippo has a first-class GraphQL editor. Set a request's **method** to `POST`, point the URL at your GraphQL endpoint, and choose **GraphQL** as the **body type**.

The query editor

The Body tab splits into two panes: the **Query** editor and a **Variables** editor for the operation's JSON variables.

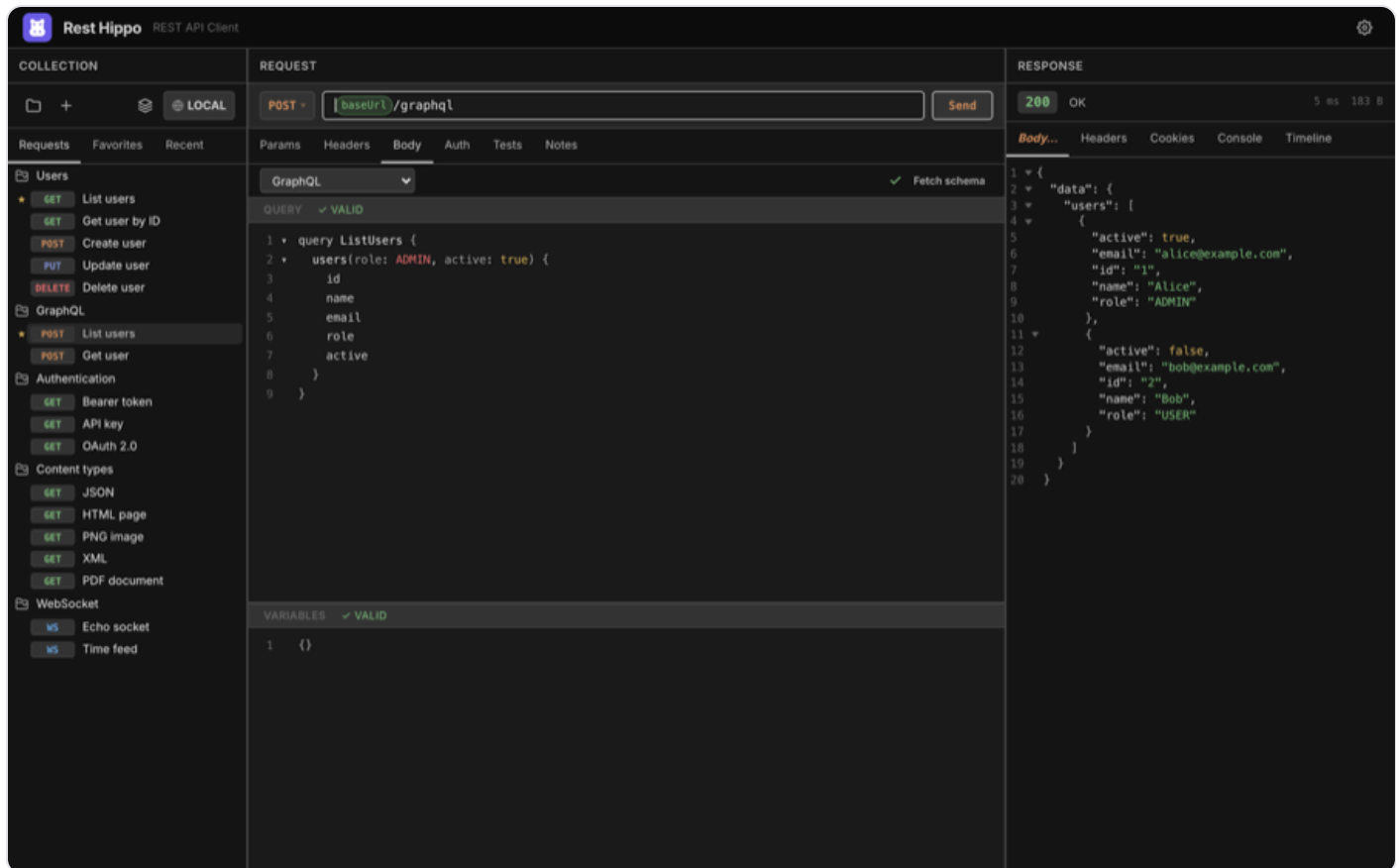


- Both panes are syntax-highlighted with line numbers and [code folding](#).
- The **Query** pane shows a **✓ VALID / ✗** badge for syntax (and, once a schema is fetched, for validity against the schema).
- The **Variables** pane validates that your variables are well-formed JSON.
- The split orientation follows the app [layout](#): side-by-side panes in wide layouts, stacked panes in tall ones. Drag the divider to resize.

You can use `{{variables}}` in the query and variables, the same as any other body.

Fetching the schema

Click **Fetch schema** to run a GraphQL **introspection** query against the request's URL. Once the schema loads, Rest Hippo uses it for two things:

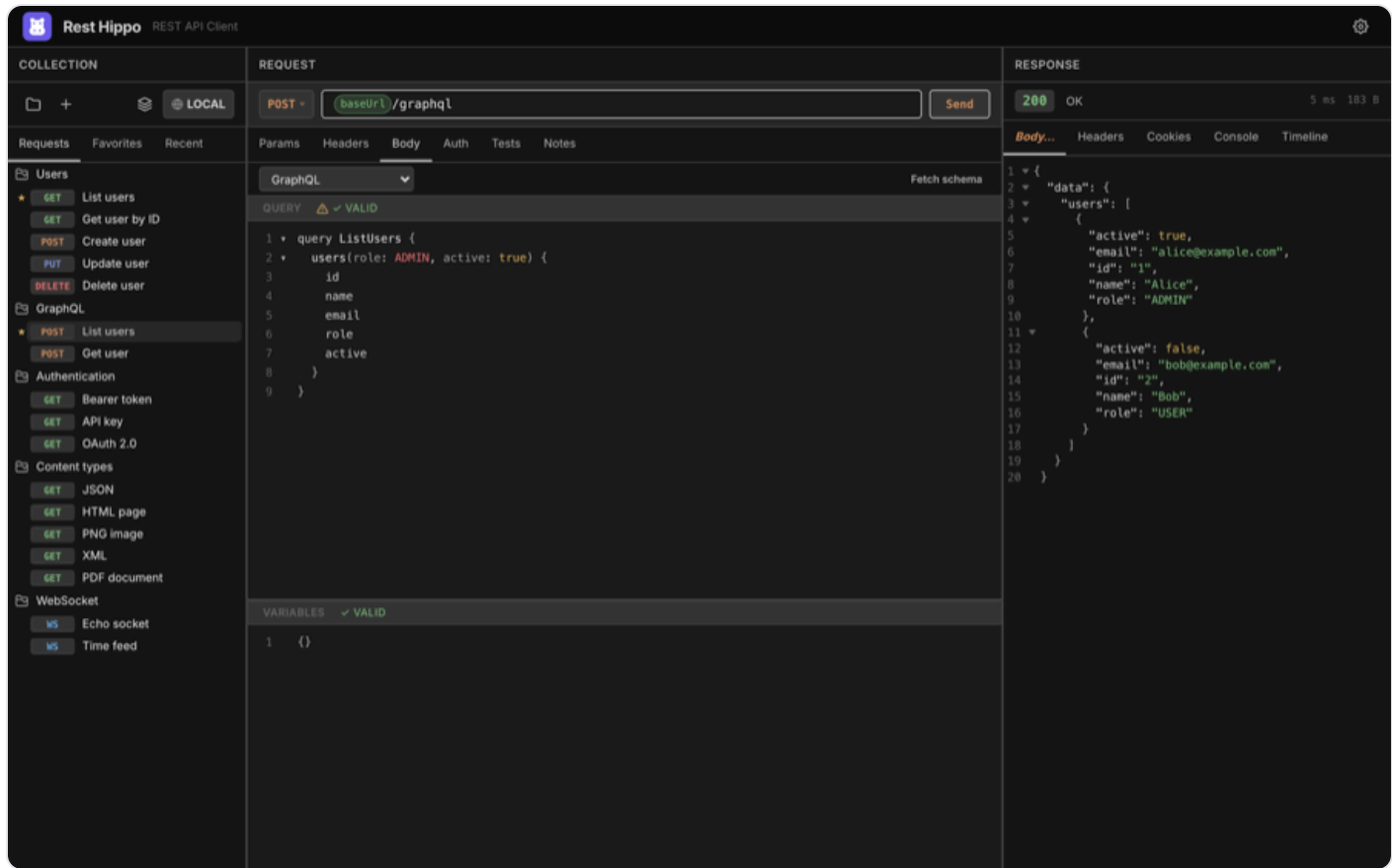


1. **Autocomplete** — typing in the Query pane suggests fields, arguments, types, and enum values from your schema.
2. **Validation** — the **✓ VALID** badge now reflects whether your query is valid *against the schema*, not just syntactically.

After a schema is loaded, the status badge shows how many types are available. **Right-click** the badge to **View Schema** (a read-only, syntax-highlighted view of the SDL) or **Download Schema** to save it as a `.graphql` file.

Sending and reading the result

Click **Send** to run the operation. The response comes back like any other — pretty-printed and syntax-highlighted in the [response viewer](#):



GraphQL servers return `200 OK` even for query errors, so check the response body's `errors` array, not just the status code.

Next: [WebSockets](#) →

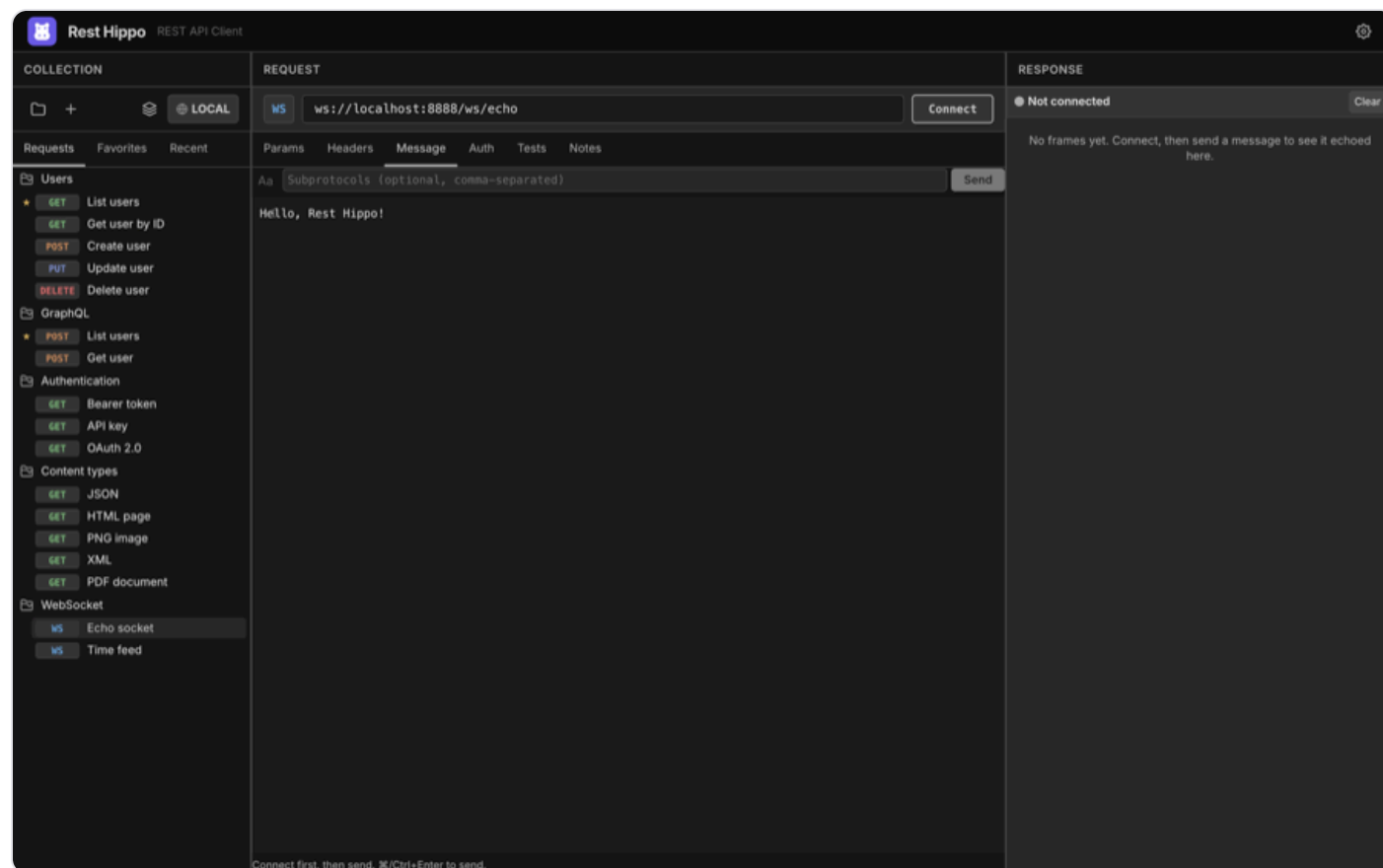
WebSockets

[← Back to contents](#)

Rest Hippo is also a WebSocket client. Create a WebSocket request from the tree (right-click a folder → **Add WebSocket Request**, or right-click the **+** button above the tree and choose **Add WebSocket Request**), and point it at a `ws://` or `wss://` URL.

The WebSocket editor

A WebSocket request looks much like an HTTP one, but the request bar shows a **WS** label and a **Connect** button, and the body is replaced by a **Message** composer:

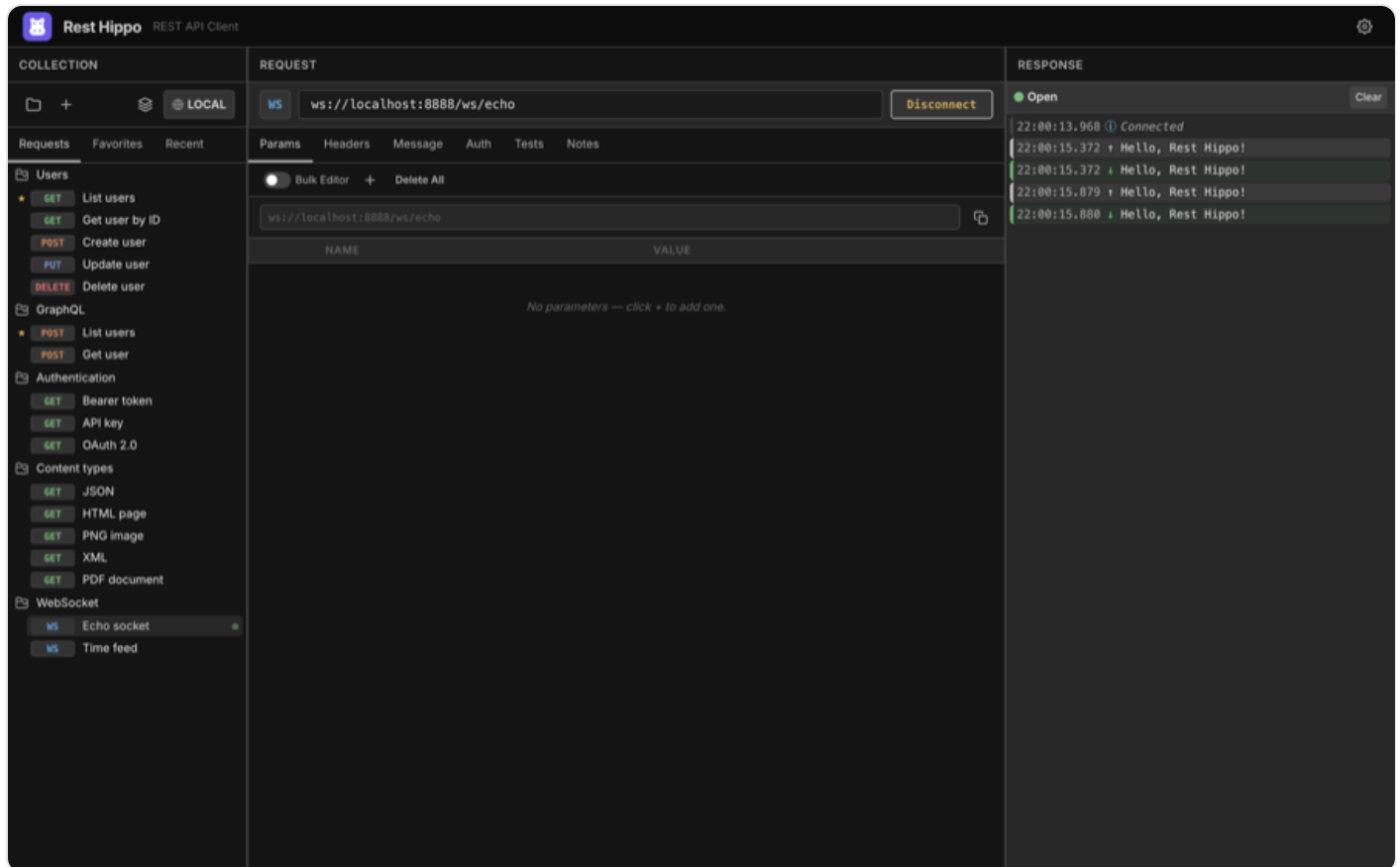


- **Message** — the frame you'll send. Toggle the format between **Text** (`Aa`) and **JSON** (`{ }`); JSON messages support `{{variables}}`.
- **Subprotocols** — an optional comma-separated list offered during the handshake.
- **Params**, **Headers**, and **Auth** apply to the opening handshake, just as they do for HTTP requests.

Connecting and sending

Click **Connect** to open the socket. The status indicator shows the connection state — *Connecting...*, **Open**, *Closing...*, **Closed**, or **Error** — and the button becomes **Disconnect**.

Once connected, edit your message and click **Send** (or press `⌘/Ctrl + Enter`). Every frame is logged in the **frame log** on the right:



The frame log is a timestamped record of the whole session:

- **Sent** frames (outgoing messages),
- **Received** frames (incoming data — including server pushes you didn't ask for), and
- **System** frames (lifecycle events: *Connected*, *Connection closed*, errors).

JSON payloads are pretty-printed automatically. Use **Clear** to reset the log; it's also reset when you connect (or reconnect) and when you switch to a different request.

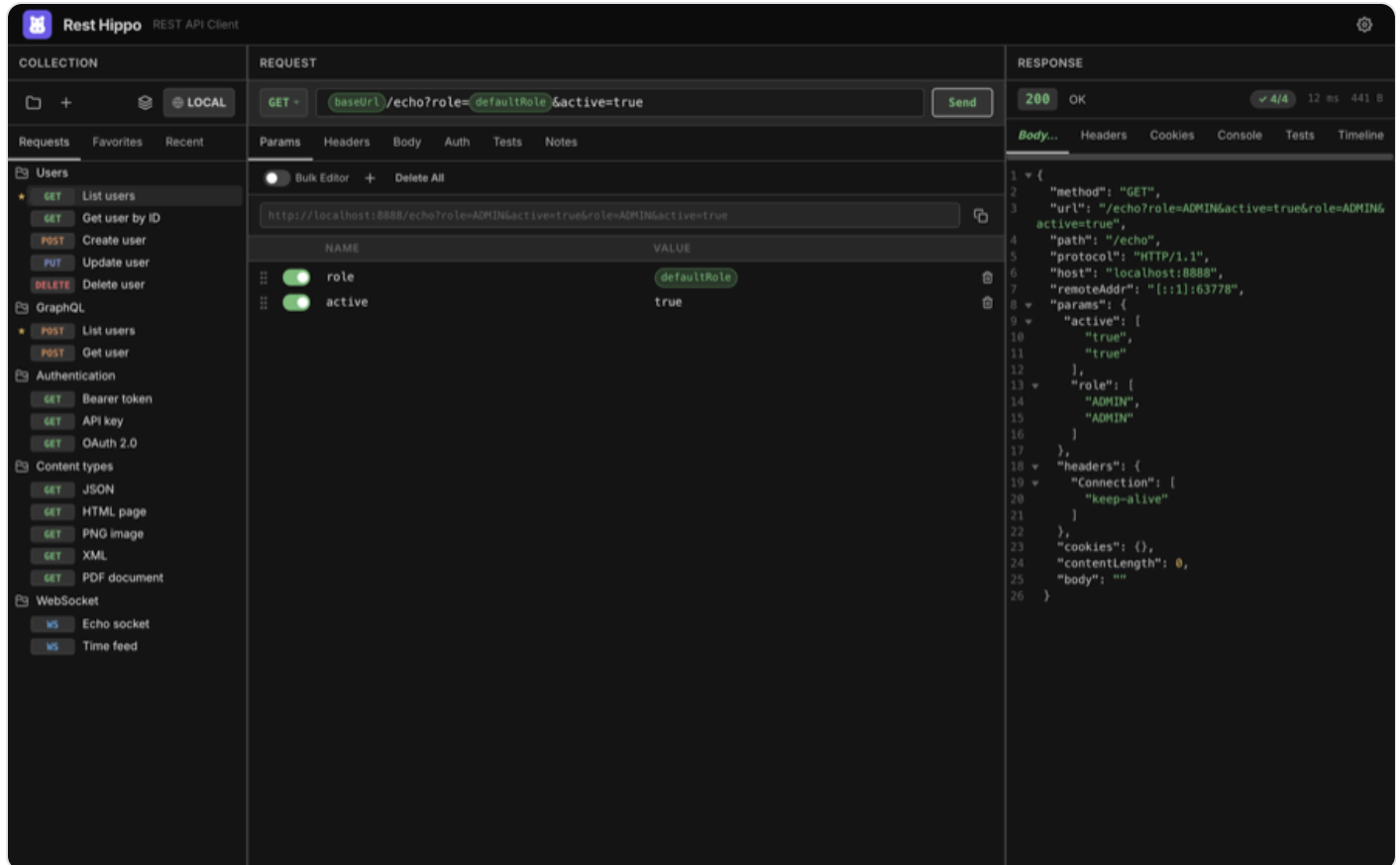
A failed handshake (for example a `401` rejection) surfaces in the log as an **Error** frame with the status, so you can tell a refused upgrade from a dropped connection.

Next: [Reading Responses](#) →

Reading Responses

[← Back to contents](#)

When a request returns, the right panel fills in. The **status bar** at the top shows the status code and text, the elapsed time, and the response size — plus a **captured** badge if any [captures](#) ran, and a green/red **test** badge if any [assertions](#) ran.



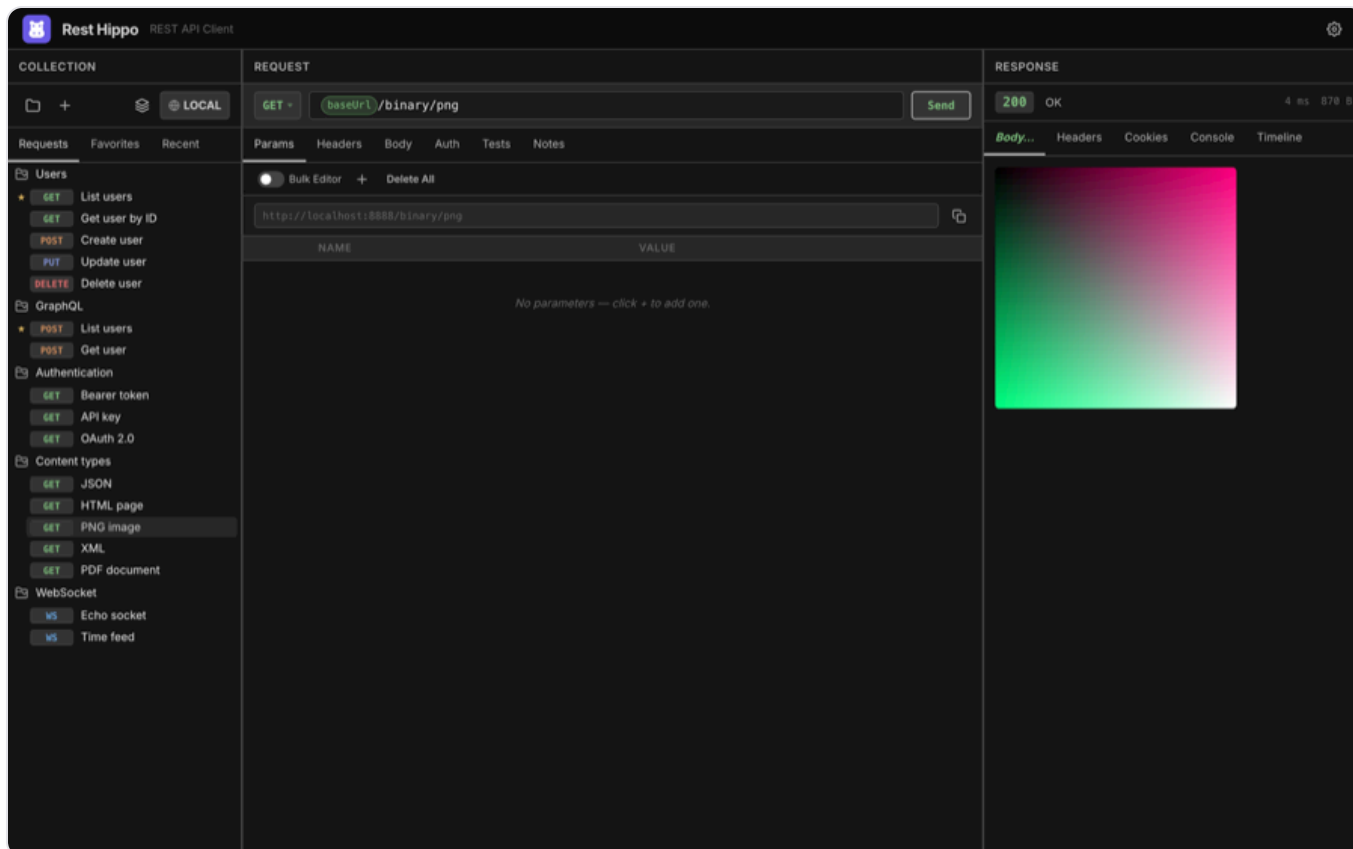
Below the status bar, a row of tabs organizes the response: **Body**, **Preview**, **Headers**, **Cookies**, **Console**, **Tests**, and **Timeline**. The **Tests** tab appears only when the run has assertions.

Body

The **Body** tab renders the response according to its content type. By default it's **Styled** — pretty-printed and syntax-highlighted:

- **JSON / YAML / XML / HTML** — indented and colored
- **Markdown** — rendered inline
- **JavaScript / CSS** — colored

- **Images** — shown inline:



- **Binary** — shown as a hex dump

Secondary-click (right-click) the `Body` tab for options: switch the render mode between **Styled**, **Raw** (plain monospace), and **Hex**, or **Download** the body (Rest Hippo picks an extension from the content type). Right-clicking the rendered body **text** itself offers **Copy** (of the current selection) plus wrap, line-number, and code-folding toggles. To reproduce the whole request as a `curl` command, use **Copy as cURL** on the request in the tree.

Preview

For **HTML** responses, the **Preview** tab renders the content live in a sandboxed view, so you can see the page as a browser would. Any `console.log` output the page produces is captured on the **Console** tab. (Markdown is rendered inline on the **Body** tab, so it has no separate Preview tab.)

PDF responses open in Rest Hippo's built-in PDF viewer with zoom and page navigation.

Streaming responses

Some endpoints send their body **incrementally** rather than all at once — Server-Sent Events (`text/event-stream`), LLM token streams, and chunked logs. Rest Hippo consumes these live: the **Body** tab becomes a timestamped, scrolling log that fills as data arrives, instead of waiting for the response to finish.

- A response whose `Content-Type` is `text/event-stream` **streams automatically** — each SSE event appears as its own row as it is received, with named events (e.g. an LLM's `content_block_delta`) tagged for clarity.
- A response whose `Content-Type` is `application/x-ndjson` streams live too, but only when you enable **Stream NDJSON responses live** under **Settings → Request**. It is off by default, so a finite NDJSON document keeps the formatted **Body** view; turn it on to watch a live NDJSON feed line by line. While the setting is off and an NDJSON request is in flight, Rest Hippo shows a brief reminder over the loading view pointing you at the setting — handy when an endless feed would otherwise just sit and buffer. The reminder clears as soon as the request finishes.

While a stream is running the toolbar shows its live **state and an event/byte counter**. You control the stream with the controls you already use elsewhere:

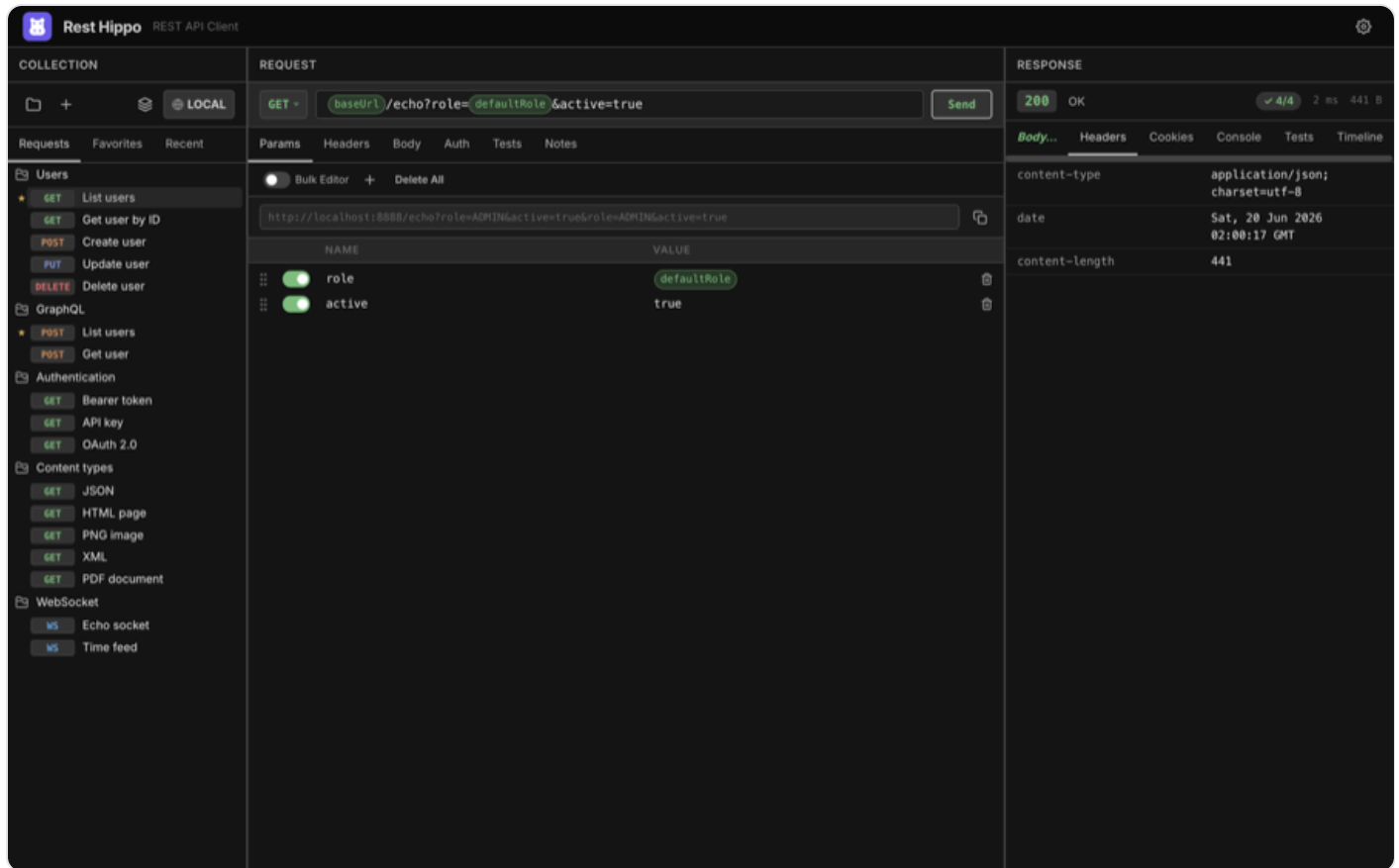
- **Stop** — the request editor's **Send** button becomes **Stop** for the whole duration of the stream, exactly like any other in-flight request. Click it to end the stream and abort the underlying request immediately.
- **Save** — right-click the response **Body** tab and choose **Download** to write the full stream to a file, including everything received so far on a stream that is still running or has been stopped.

The on-screen log is capped so a long-running (or never-ending) stream stays memory-bounded; the complete stream is always available through the Body tab's **Download** menu item.

The live log itself is session-scoped and isn't persisted, but **each streaming run leaves a record in the Timeline** — when it was sent, how long it ran, how many events and bytes arrived, and the last few events received. Reopening that record shows the summary in place of a body, so you can review a past stream even though its full payload isn't kept.

Headers

The **Headers** tab lists every response header as a name/value table:

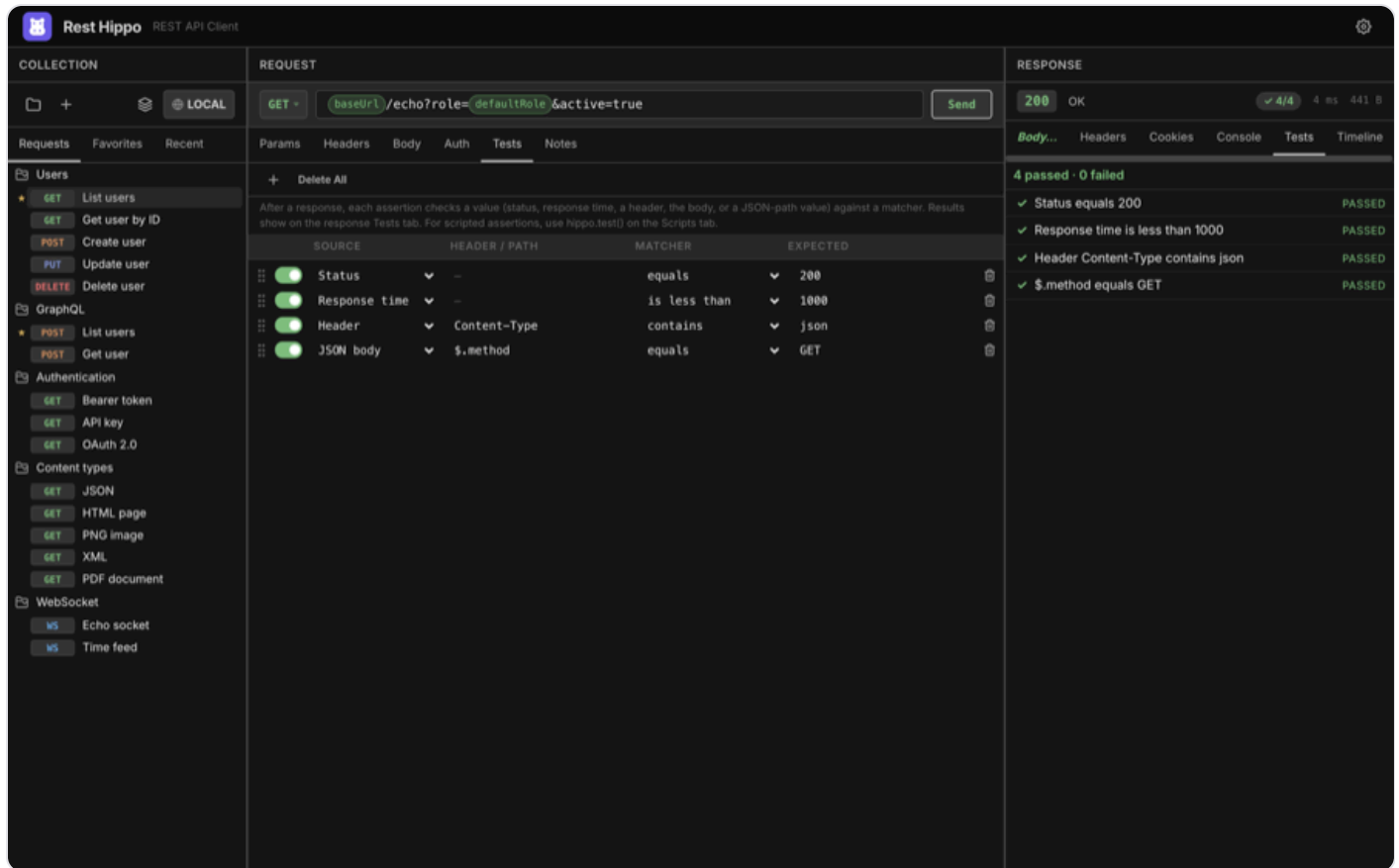


Cookies

The **Cookies** tab parses the response's `Set-Cookie` headers into a table — name, value, domain, path, and the `Secure` / `HttpOnly` / `SameSite` / `Expires` attributes. If the collection has [cookie sending](#) enabled, these cookies are stored and reused on later requests.

Tests

The **Tests** tab shows the result of any **assertions** attached to the request — a quick way to turn a request into an API check. Each assertion appears with a **✓ pass** or **✗ fail** marker, its name, and (on failure) the reason it failed; a summary line at the top counts how many passed and failed. The status bar also shows a compact pass/fail badge, green when everything passed and red when anything failed, so you can tell at a glance without switching tabs.



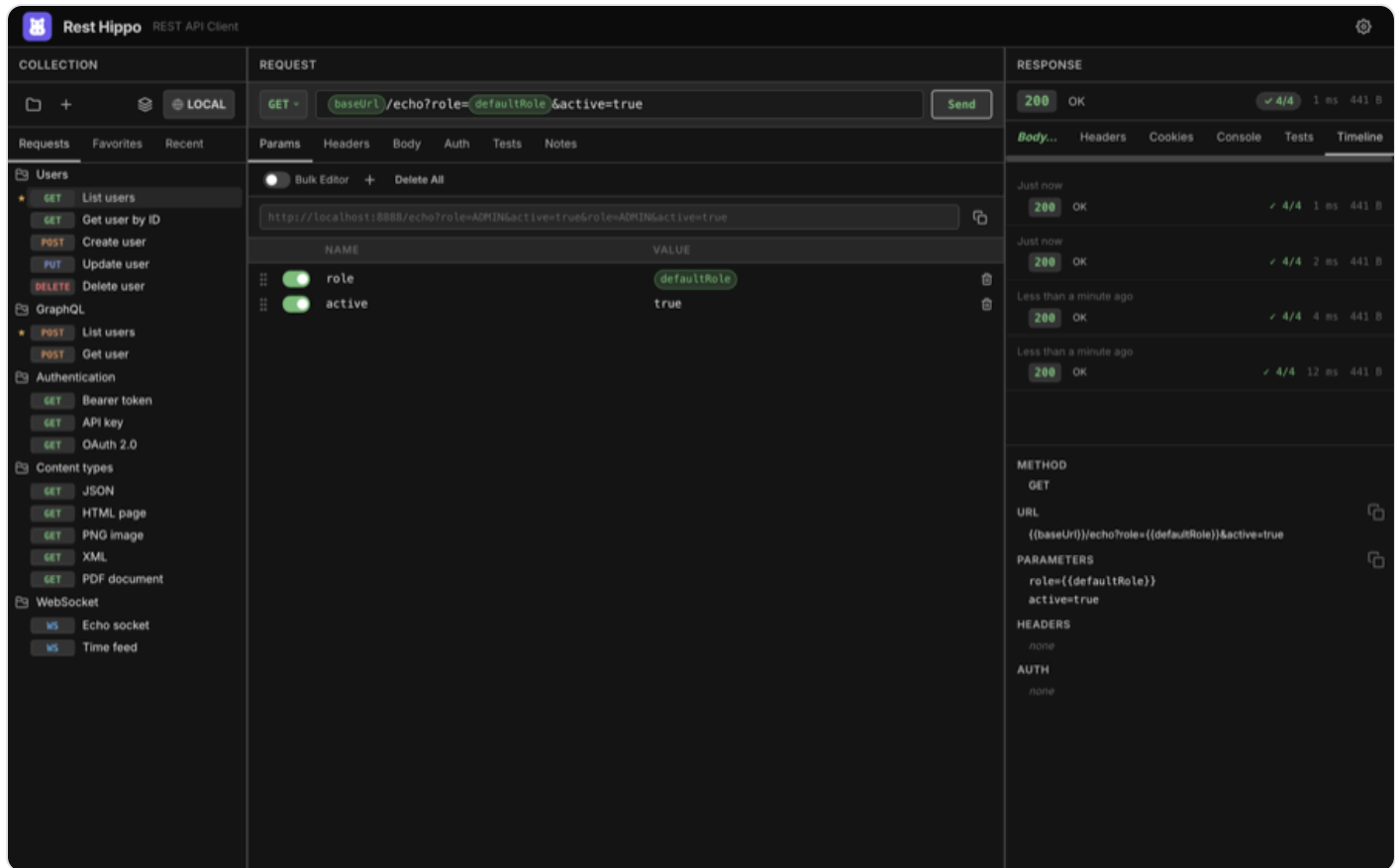
There are two ways to author assertions, and both run after the response arrives and feed the same Tests tab:

- **No-code grid** — the **Tests** request tab (enable it under **Settings** → **Request** → **Show Tests tab**). Add a row per check: pick a **source** (status, response time, a header, the body, or a JSON-path value), a **matcher** (equals, contains, exists, less than, matches, ...), and an **expected** value.
- **Scripted** — `hippo.test()` in the after-response pane of the **Scripts** tab, for checks that need logic.

Results are saved with the run, so reopening a past entry in the **Timeline** shows whether that run passed.

Timeline

The **Timeline** tab keeps a short history of the request's recent runs. Each entry records the status, timing, a pass/fail count for any assertions that ran, and a snapshot of what was sent — method, URL, parameters, headers, and auth — so you can compare runs and reopen an earlier response. **Streaming runs** appear here too, recorded as a summary (duration, event/byte counts, and the last events) rather than a body:

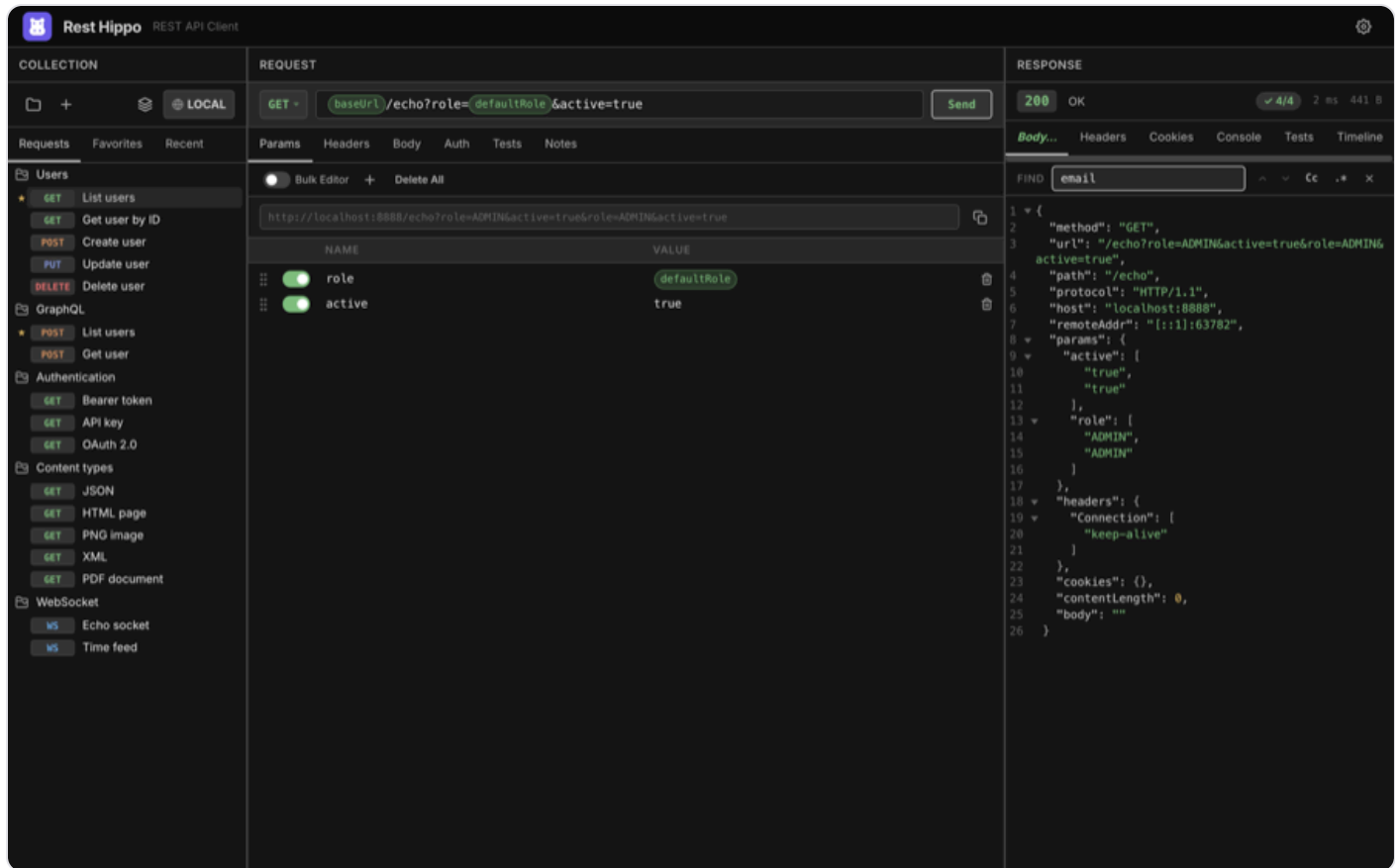


- **Click** an entry to view that run — its response loads into the Body, Headers, and Cookies tabs and its snapshot shows on the right. This is non-destructive; your live request is untouched.
- **Double-click** an entry (or right-click → **Restore Into Editor**) to replay that snapshot back into the request editor.
- **Right-click** for the full menu: restore, delete the entry, or clear the whole history.
- **Right-click the Timeline tab** itself to switch to it and get a quick **Delete All History** — the same as clearing from an entry's menu.

How many runs are kept is configurable in [Settings → History](#).

Searching the body

Press `⌘/Ctrl + F` with the response focused to open the **Find** bar:



- Type to highlight matches; the counter shows the active match and the total.
- `Enter` / `Shift` + `Enter` jump to the next/previous match.
- Toggle **case-sensitive** and **regex** matching.
- `Esc` closes the bar.

`⌘/Ctrl` + `A` selects the whole body text (when the find box isn't focused).

Filtering the body

Press `⌘/Ctrl` + `Shift` + `F` with the response focused to open the **Filter** bar. Where Find *highlights* matches, Filter *transforms* the body to just the fields your expression selects, re-rendered in the same styled view. The expression language depends on the body's type:

- **JSON** — a `jq` filter, e.g. `.items[] | select(.active) | .name`.
- **YAML** — the same `jq` filter applied to the parsed document (like `yq`); the result is re-emitted as YAML.
- **XML** — an `XPath` expression, e.g. `//book/title`.

Type to filter as you go; an invalid expression turns the box red and leaves the original body untouched.

`Esc` or the **X** button closes the bar and restores the full response.

Filtering is only offered for a **styled** JSON, YAML, or XML body. If the body is shown in Raw or Hex mode, or is some other content type, a notification explains that the current MIME type or view doesn't support filtering.

Next: [Scripts](#) →

Scripts

[← Back to contents](#)

The **Scripts** tab on a request lets you run a little JavaScript **before** the request is sent and **after** the response comes back. Use it to prepare the outgoing request (set a header, rewrite the URL), to pull a value out of the response and stash it in a [variable](#) for a later request, or just to log something while you debug.

There are two panes:

- **Pre-request** — runs just before the request is sent. It can change the outgoing request and set variables.
- **After-response** — runs as soon as the response arrives. It can read the response and set variables.

Both panes are full code editors with JavaScript highlighting; syntax errors are underlined as you type.

Each pane header has an **Enabled** toggle — a script runs only when its pane is enabled, so you can keep a script on a request but switch it off without deleting it. Drag the divider between the two panes to resize them; the split is saved with the request.

When scripts run

```
Pre-request script → {{variables}} substituted → request sent
                                     ↓
                               variables / UI updated ← After-response script
```

Because the pre-request script runs **before** variable substitution, a variable you set with `hippo.variables.set(...)` is available to the very request you are sending — `{{thatVariable}}` resolves with the new value. Variables set in either pane are also saved, so a **later** request can use them too.

The `hippo` API

Everything a script can do lives under the global `hippo` object. The surface is small and deliberately sandboxed (see [Sandbox & limits](#)).

Member	Parameters	Returns	Available in	Notes
<code>hippo.variables.get(scope, name)</code>	<code>scope :</code> <code>"global" </code> <code>"environment" </code> <code>"folder" ·</code> <code>name : string</code>	<code>string undefined</code>	both	Reads a variable from a specific scope. (A legacy <code>"collection"</code> is treated as <code>"global"</code> .)
<code>hippo.variables.set(scope, name, value)</code>	<code>scope :</code> <code>"global" </code> <code>"environment" ·</code> <code>name : string ·</code> <code>value : string</code>	<code>void</code>	both	Writes a variable; the value is saved for later requests. <code>"folder"</code> is read-only — <code>set</code> throws. (A legacy <code>"collection"</code> is treated as <code>"global"</code> .)
<code>hippo.request.method ·</code> <code>.url · .headers · .body</code>	—	<code>string ·</code> <code>string ·</code> <code>object ·</code> <code>string</code>	both	Mutable in the pre-request script (changes the outgoing request); a read-only snapshot in the after-response script. In the pre-request pane these still contain <code>{{templates}}</code> — they are substituted after your script runs.
<code>hippo.response.status ·</code> <code>.time · .headers · .body</code>	—	<code>number ·</code> <code>number (ms) ·</code> <code>object ·</code> <code>string</code>	after-response only	Reading these in a pre-request script throws — the response doesn't exist yet. <code>.time</code> is the elapsed response time in milliseconds.
<code>hippo.response.json()</code>	—	<code>parsed value</code>	after-response only	Parses the response body as JSON; throws if the body isn't valid JSON.
<code>hippo.environment</code>	—	<code>{ name,</code> <code>variables</code> <code>}</code>	both	The active environment (read-only).

Member	Parameters	Returns	Available in	Notes
<code>hippo.run(requestName)</code>	<code>requestName</code> : string literal	<code>{ status, time, headers, body, json() }</code>	both	Runs another saved request by name and returns its response (same shape as <code>hippo.response</code>). The name must be a string literal — see Run another request .
<code>hippo.console.log</code> / <code>.info</code> / <code>.warn</code> / <code>.error</code>	<code>...args</code> : any	<code>void</code>	both	Logs to the response Console tab (see Console output).
<code>hippo.test(name, fn)</code>	<code>name</code> : string · <code>fn</code> : function	<code>void</code>	after-response	Runs <code>fn</code> as a named test; if it throws, the test fails (see Test assertions).
<code>hippo.expect(value)</code>	<code>value</code> : any	matcher	after-response	Returns a matcher with <code>.toBe</code> / <code>.toEqual</code> / <code>.toContain</code> / <code>.toBeLessThan</code> / <code>.toBeGreaterThan</code> / <code>.toMatch</code> / <code>.toBeTruthy</code> / <code>.toBeFalsy</code> (and <code>.not</code>), each throwing on mismatch.

Scopes follow the same precedence as everywhere else in Rest Hippo: folder → environment → global. `get` can read any of the three; `set` writes to global or environment (folder variables are read-only from a script). Global is the collection-wide tier.

Example: prepare the request (pre-request)

```
// Build an Authorization header from an environment variable.
const key = hippo.variables.get("environment", "apiKey");
hippo.request.headers["Authorization"] = "Bearer " + key;

// Or set a variable and reference it as {{token}} in the URL / headers – it is
// substituted into THIS request because the script runs before substitution.
hippo.variables.set("environment", "token", "abc123");
```

Example: run another request first (pre-request)

`hippo.run("Name")` runs another saved request and hands you back its response, so a script can chain a dependency — log in, grab a token, and use it on the request you are about to send:

```
// Log in first, then carry the token on this request.
const res = hippo.run("Login");
if (res.status === 200) {
  hippo.request.headers["Authorization"] = "Bearer " + res.json().token;
}
```

The returned response has the same shape as `hippo.response`: `status`, `time` (ms), `headers`, `body`, and `json()`.

The request name must be a **string literal** — Rest Hippo finds and runs it *before* your script does, so a computed name like `hippo.run(someVariable)` can't work and throws. The named request runs with its own saved settings but does **not** run its own pre- or after-response scripts, so two requests can't recursively trigger each other. `hippo.run` works the same way in an after-response script.

Duplicate names. `hippo.run("Name")` matches **by name**, so if two saved requests share that name it runs the first one and shows a warning — give them distinct names to be sure. The cross-request **function pills** you insert into a field (`run`, `response`, `responseHeader`, `responseStatus`) don't have this problem: the picker stores the request's **stable id**, so they keep pointing at the same request even if you rename it, and the picker shows each request's folder path when a name is duplicated so you can tell them apart.

Example: capture from the response (after-response)

```
// On success, pull an id out of the JSON body and save it for a later request.
if (hippo.response.status === 200) {
  const body = hippo.response.json();
  hippo.variables.set("global", "userId", body.id);
  hippo.console.log("captured userId", body.id);
}
```

A following request can now use `{{userId}}` in its URL, headers, or body. (For simple field extraction without writing code, see also [Captures](#).)

Test assertions

An after-response script can also **validate** the response, turning a request into an API check. Use `hippo.test(name, fn)` to declare a named test and `hippo.expect(value)` for the comparison — if the matcher (or anything inside the test) throws, the test is marked failed; otherwise it passes. Results appear on the response [Tests](#) tab with a pass/fail badge in the status bar.

```
hippo.test("status is 200", () => {
  hippo.expect(hippo.response.status).toBe(200);
});

hippo.test("responds quickly", () => {
  hippo.expect(hippo.response.time).toBeLessThan(500);
});

hippo.test("returns the right user", () => {
  const body = hippo.response.json();
  hippo.expect(body.id).toBe(42);
  hippo.expect(body.roles).toContain("admin");
});
```

Matchers: `toBe` (strict equality), `toEqual` (deep equality), `toContain` (substring / array member), `toBeLessThan` / `toBeGreaterThan`, `toMatch` (regex), `toBeTruthy` / `toBeFalsy`. Prefix any with `.not` to invert it, e.g. `hippo.expect(hippo.response.status).not.toBe(500)`.

Prefer not to write code? The no-code **Tests** tab lets you build the same checks from a grid of source → matcher → expected rows. Grid assertions and scripted `hippo.test()` calls run together and share the one Tests tab.

Console output

`hippo.console.log/info/warn/error(...)` writes lines to the response **Console** tab, tagged so you can tell them apart from the HTTP verbose log. This is the easiest way to see what a script is doing — log a value, send the request, and read it back on the Console tab.

Errors

If a script throws, Rest Hippo surfaces it — it is never swallowed:

- An error toast shows the message and the line number.
- The offending line is marked with a red squiggle in the editor.
- A **pre-request** error **cancels the send** — a half-prepared request is never sent. Any variable writes from that run are discarded.

Sandbox & limits

Scripts run in a locked-down sandbox in Rest Hippo's main process. They **cannot** reach the filesystem, the network, `process`, or `require`, and `eval` / the `Function` constructor are disabled. A script that loops forever is stopped by a one-second timeout.

Scripts are **synchronous** — there is no `await` (there's nothing to wait for inside the sandbox). Keep them small: read a value, set a variable, log something. Even `hippo.run("...")` is synchronous: Rest Hippo runs the named request *outside* the sandbox before your script starts, then hands the response in — which is why the request name has to be a literal it can spot ahead of time.

Next: [Import, Export & Backup](#) →

Import, Export & Backup

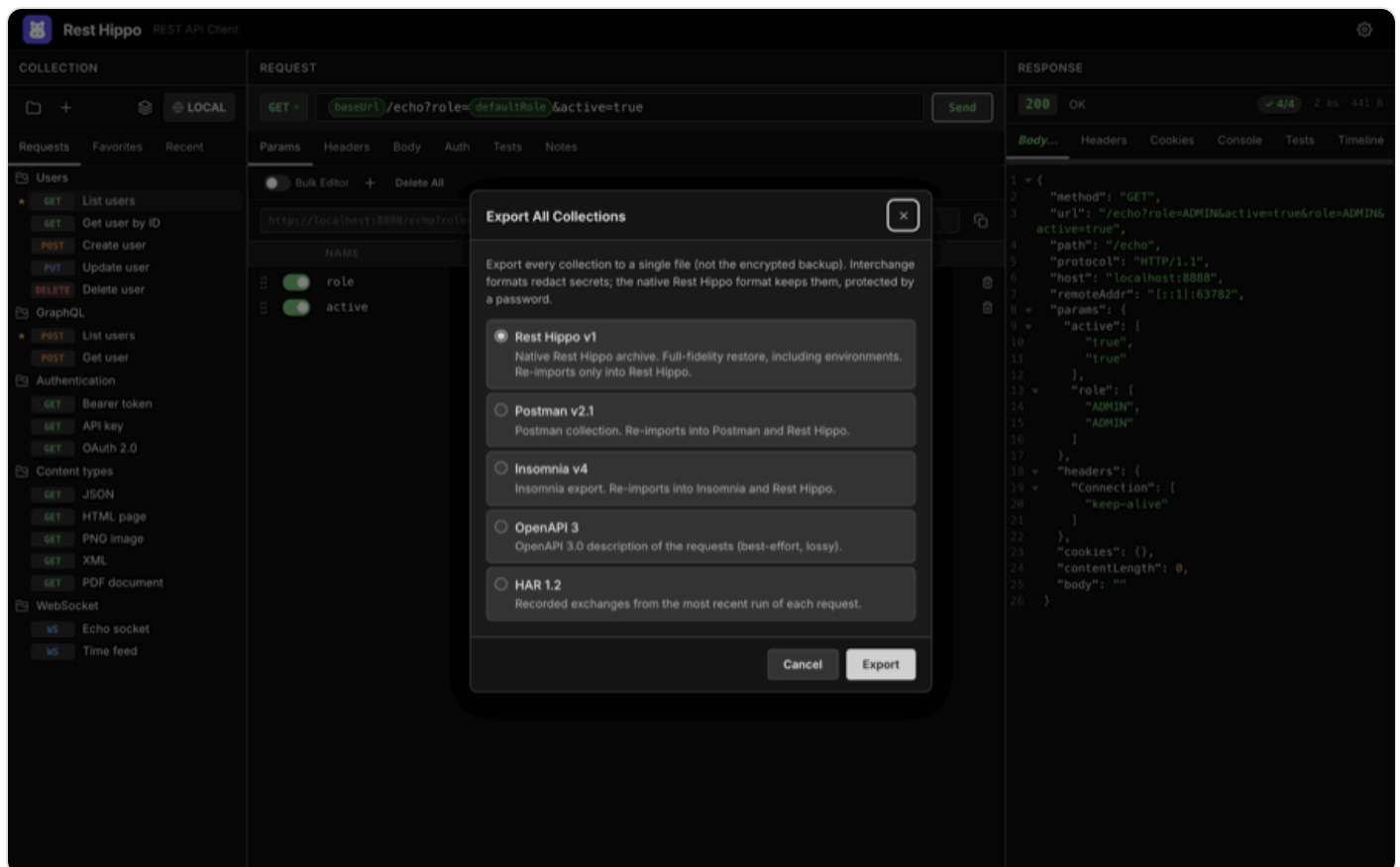
[← Back to contents](#)

Rest Hippo can exchange collections with other tools and back up your entire workspace. There are two separate features:

- **Import / Export** moves *collections* in and out. The native **Rest Hippo v1** format is a lossless, full-fidelity archive (secrets kept, password-protected) that restores in place; the standard interchange formats (Postman, Insomnia, OpenAPI, HAR) are for sharing with other tools and **redact** secrets.
- **Backup / Restore** snapshots your *whole workspace* — collections, environments, and settings — with a choice of how to handle secrets.

Exporting

Export a single collection from its right-click menu (**Export...**), or export everything at once — from the **File > Export All Collections...** menu or the export button in the [collections manager](#) toolbar:



Choose a format:

Format	Notes
Rest Hippo v1	Native, lossless archive. Restores everything exactly — including environments. Re-imports only into Rest Hippo.
Postman v2.1	Postman collection. Re-imports into Postman and back into Rest Hippo.
Insomnia v4	Insomnia export. Re-imports into Insomnia and back into Rest Hippo.
OpenAPI 3	A best-effort, lossy OpenAPI 3.0 description of the requests.
HAR 1.2	Recorded request/response exchanges from recent runs.

In the **interchange** formats (Postman, Insomnia, OpenAPI, HAR), secrets — passwords, tokens, and keys — are **redacted**, so those exports are safe to share. Rest Hippo then opens a native save dialog.

The native Rest Hippo format

Rest Hippo v1 is a full-fidelity archive meant for round-tripping your own work — moving a collection between machines, or snapshotting it before a big edit. Unlike the interchange formats, it keeps **everything** about the exported requests: method, URL, query and path params, headers, the full **Auth** configuration for every scheme, captures, scripts, notes, and bodies.

- **Variables travel referenced-only.** The archive adds an *environments* section (the collection's environments and global variables) and the collection's own variables — but only the ones the exported requests reference (`{{name}}` , followed transitively). Anything unused is left out, so exporting a single folder never drags in the rest of the collection's variables (or forces a password for secrets it doesn't use). A folder's own variables always travel inside that folder.
- **Secrets are kept, not redacted.** If the exported set contains any secret it actually uses (an auth credential or a referenced `secure` variable with a value), Rest Hippo asks you to choose a password and encrypts those values into the file with the same scheme as a password-protected backup. With no secrets, there's no prompt and the file is plain JSON. **Treat a password-protected archive like a credential export** — anyone with the file *and* the password can read its secrets.
- **Import merges in place.** Re-importing restores into your active collection rather than creating a duplicate: a folder that already exists (matched by id, then name) is reused and its contents restored into it; a request that already exists is **replaced** by the archived copy; anything new is created. Environments and variables are matched the same way and only *added* when missing — an existing value is never overwritten. A password-protected archive prompts for its password on import.

Importing

Open **File → Import Collection...** and pick a file. Rest Hippo recognizes the format automatically:

- **Rest Hippo v1** native archives (`.json`) — merged in place (see above)
- **Postman** collections (`.json`)
- **Insomnia** exports (`.json` / `.yaml`)
- **OpenAPI 3** / Swagger 2.0 specifications (`.json` / `.yaml`)
- **HAR 1.2** captures (`.har`)

The file picker filters to these formats — `.json` , `.yaml` / `.yml` , and `.har` . On macOS, hovering the **Import Collection...** menu item also shows this list as a tooltip (native menu tooltips are a macOS-only feature, so on Windows and Linux this page is the reference).

It reconstructs the folder structure, requests, headers, query, auth, and variables, and adds them to your workspace as a new collection. A **HAR** capture (a browser's "Save all as HAR", or a proxy export) is imported request by request — grouped into a folder per host — so you can replay real traffic; only the requests are imported, not the recorded responses.

OpenAPI / Swagger specs describe their operations as relative paths, so importing one first asks for a **base-URL variable**: a name (default `baseUrl`) and a value. Every imported request is prefixed with that variable — e.g. `{{baseUrl}}/pets/{{petId}}` — so the host lives in one editable place. The value is pre-filled from the spec's `servers` / `host` when it has one; otherwise leave it blank and set it later in the collection's **Variables**.

Import from cURL

To pull in a single request from a terminal, API docs, or a browser's **Copy as cURL**, choose **File → Import from cURL...** and paste the command:

```
curl https://api.example.com/users \
  -H 'Authorization: Bearer ...' \
  -d '{"name":"Ada"}'
```

Rest Hippo parses the method, URL and query, headers, body (`-d` / `--data*` , `--data-urlencode` , and `-F` form fields), and authentication — a `-u user:pass` or an `Authorization: Bearer / Basic` header is lifted into the request's **Auth** tab rather than left as a raw header. The result is added as a new collection, ready to send.

Tip — paste a cURL straight onto a request. You can also paste a `curl ...` command directly into a request's **URL bar**. Rest Hippo recognizes it and rewrites that request to match the command (method, URL, params, headers, body, auth), instead of dropping the raw text into the field. A brand-new, empty request is updated in place; if the request already has content, Rest Hippo asks you to confirm before overwriting it.

Import from a URL

When a document is published online, you don't have to download it first — choose **File → Import from URL...** and paste its address:

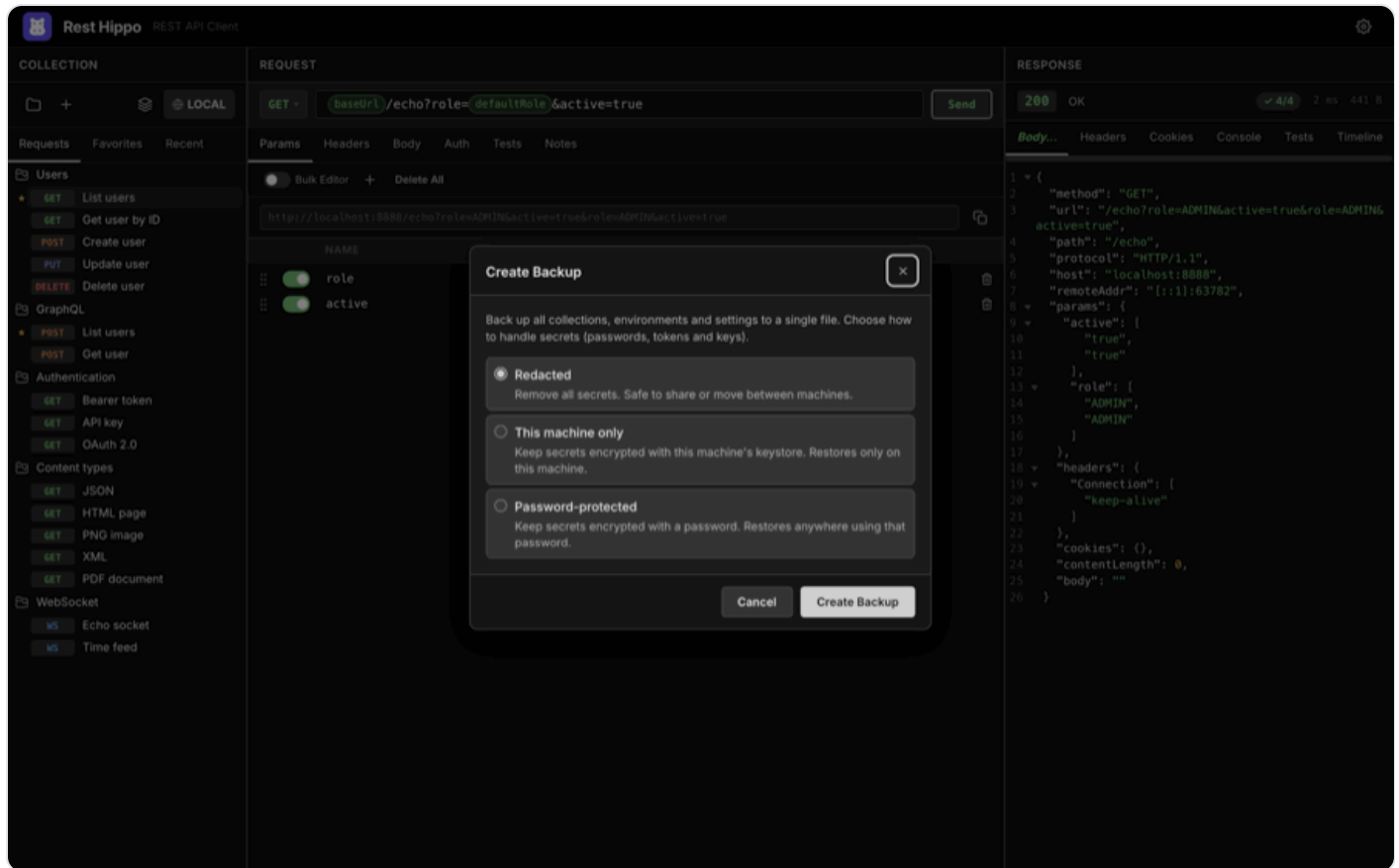
```
https://api.example.com/openapi.json
```

Rest Hippo fetches the document through the desktop app's request engine (not the browser), so there are no CORS limits, and a document served from `http://localhost` by a service running on your machine works just as well. If it sits behind a token, add the optional **Authorization header**: a bare value such as `Bearer ...` is sent as `Authorization`, or use a `Name: Value` line for a custom header (e.g. `X-API-Key: ...`).

The format is detected automatically — the same set as a file import (Postman, Insomnia, OpenAPI 3 / Swagger 2.0, HAR) — and the import then continues exactly like one. An **OpenAPI / Swagger** spec still shows the **base-URL variable** prompt described above, with its value pre-filled from the host you imported from (so a spec with a relative `servers` path still gets a complete, usable base URL); every other format imports straight through with no prompt.

Backup & restore

A **backup** captures your complete workspace in one file. Open it from the app's File menu (**Back up... / Restore...**).



When creating a backup, choose how secrets are handled:

Mode	Secrets	Restores on...
Redacted	Removed entirely	Anywhere — safe to share or move between machines
This machine only	Kept in their at-rest encrypted form, tied to this machine's Security setup	Only this machine
Password-protected	Encrypted with a password you choose	Anywhere, with the password

Restoring reads a backup file (prompting for the password if it's password-protected) and lets you **Merge** it alongside your current collections or **Replace** everything with the backup's contents.

Use **This machine only** for routine local backups (no password to remember), and **Password-protected** when you need to move a full workspace — secrets and all — to another machine.

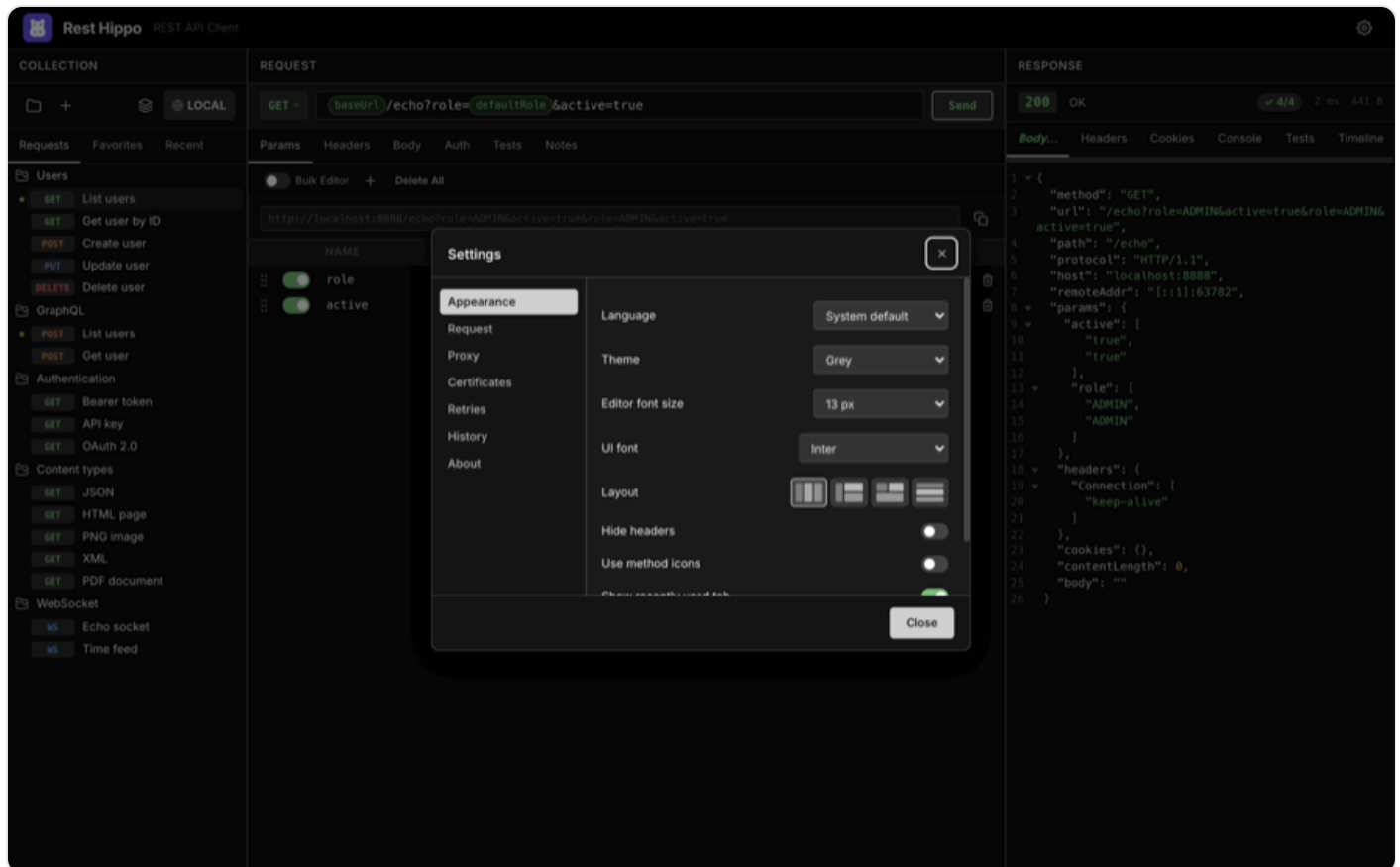
Next: [Settings & Themes](#) →

Settings & Themes

[← Back to contents](#)

Open **Settings** from the ⚙ button in the header. Settings are grouped into panels down the left side and **save as you change them** — there's no separate save step.

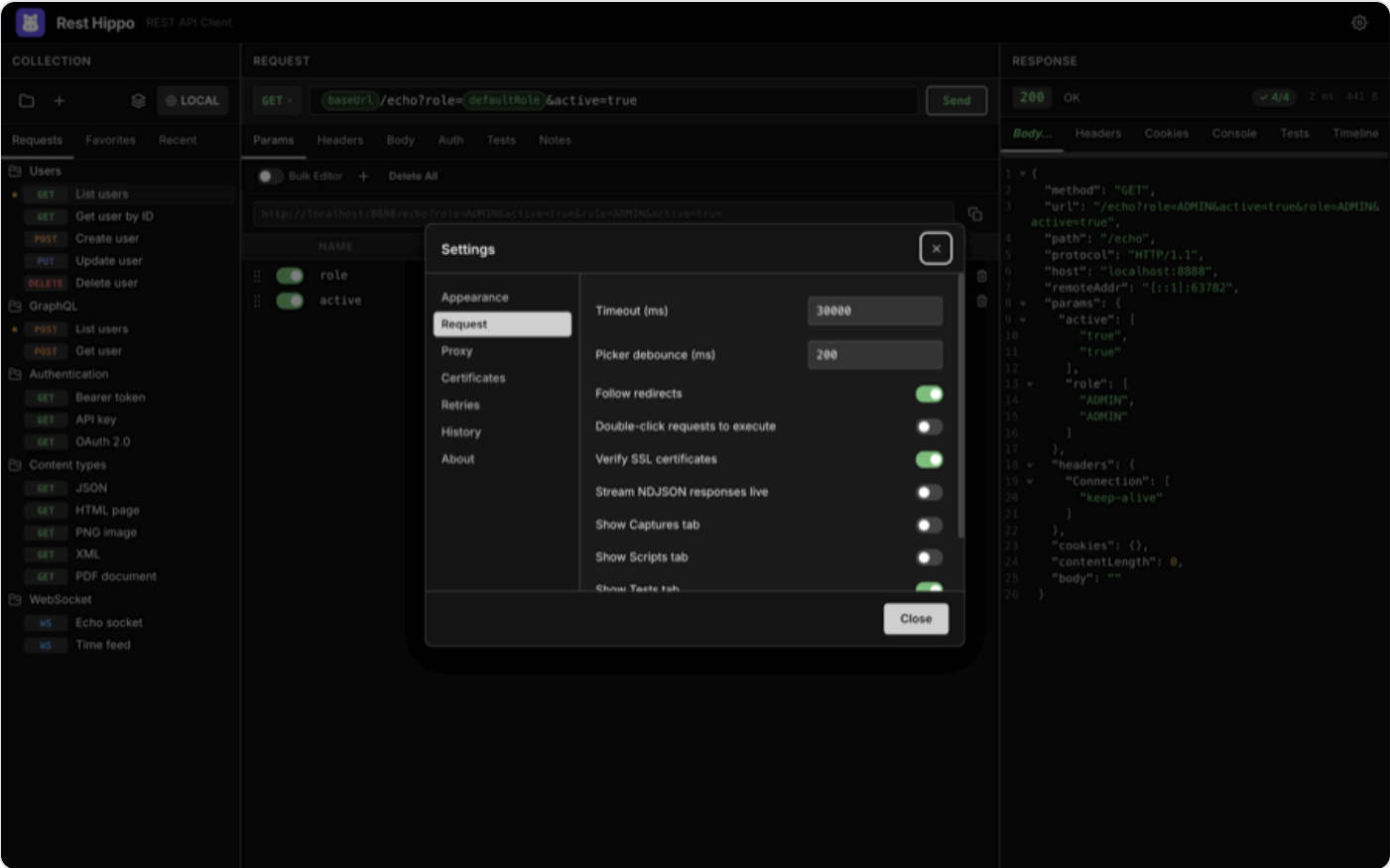
Appearance



Setting	What it does
Language	The interface language — System (follow the OS) or English, Deutsch, Español, Français, Italiano, , . Changing it reloads the window.
Theme	The color theme, plus the Theme Editor... entry (see Themes).
Editor font size	Size of text in the code editors (11, 12, 13, 14, 16, or 18 px).
UI font	The interface typeface — Inter (bundled), your system font, or SF Pro / Segoe UI / Ubuntu / Roboto.
Layout	The panel layout (1–4).
Show headers	Shows the top header bar. Turn it off to hide the bar and move its controls into the collections panel, for a more compact window.
Use method icons	Show HTTP method badges as compact icons instead of text.
Show recently used tab	Show or hide the Recent tab.
Show URL preview	Show the resolved URL (with <code>{{variables}}</code> filled in) directly beneath the URL bar.

You can change the editor font size anywhere with `⌘/Ctrl + + / - / 0`. **Code folding** is toggled from each editor's own right-click menu rather than from Settings.

Request



Setting	What it does
Timeout (ms)	How long to wait before giving up on a request.
Picker debounce (ms)	Delay before the <code>{{</code> typeahead appears.
Follow redirects	Automatically follow 3xx redirects.
Double-click requests to execute	Double-clicking a request in the tree loads <i>and</i> sends it.
Verify SSL certificates	Reject invalid/self-signed certificates. Turn off to test against dev servers.
Show Captures tab	Show the Captures tab on the request editor. When off, the tab is hidden <i>and</i> capture rules are not run after a response. Off by default.
Show Scripts tab	Show the Scripts tab on the request editor. When off, the tab is hidden <i>and</i> pre-request / after-response scripts are not executed. Off by default.
Show Tests tab	Show the Tests tab on the request editor. When off, the tab is hidden <i>and</i> no-code assertions are not run after a response. Off by default.
Show Notes tab	Show the Notes tab on the request editor. On by default.

Proxy

Route requests through an HTTP/HTTPS or SOCKS proxy. Enter a **Proxy URL** (the scheme — `http://` , `socks5://` , ... — selects the proxy type), optionally enable **proxy authentication** with a username and password, and list hosts to **bypass** (a `NO_PROXY` -style list supporting suffixes and `*` globs).

Certificates

Configure mutual TLS (mTLS) and custom trust for hosts that need more than the system certificate store.

- **Client certificates** — present a certificate to hosts that require mutual TLS. Each entry has a **Host** pattern (exact, suffix, or `*` glob, with an optional `:port` , matched like the proxy bypass list) and a **Format**: choose **PEM** to point at a certificate file plus an optional private-key file, or **PFX / P12** for a single bundle. A **passphrase** can be supplied for an encrypted key or PFX. The first entry whose host matches a request wins, so list a specific host above a broader wildcard. The certificate is also presented automatically on redirects to a matching host and on OAuth token requests to one.
- **Certificate authorities** — add CA files to trust **in addition to** the system roots. This lets a privately-signed host validate with verification still on, instead of turning verification off globally.

- **Skip verification for hosts** — a `NO_PROXY` -style list of hosts whose TLS certificate is not checked. Use this only for trusted self-signed hosts when a custom CA isn't practical; it overrides the global **Verify SSL** toggle for those hosts only.

Only file **paths** are stored — Rest Hippo reads the certificate bytes in the background process when a request is sent. Passphrases are encrypted at rest using the secret-storage method chosen under **Security** (like all other secrets) and are removed from secret-free exports.

Security

Choose how Rest Hippo encrypts your secrets — auth passwords and tokens, OAuth client secrets, proxy credentials, mTLS passphrases, and variables marked **secure** — at rest on this device. Switching re-encrypts every stored secret; the change applies after a quick reload.

- **This device (no prompt)** — the default. Secrets are encrypted with a random key kept in a protected file on this device, so there are **no system prompts**. The trade-off: the key sits alongside your data, so anyone who can read this computer's files could read your secrets. (On Windows and on Linux without a keyring, the OS-keychain option falls back to this behaviour.)
- **OS keychain** — secrets are encrypted with your operating system's keychain (macOS Keychain, Windows Credential Manager, Linux Secret Service). This is the strongest at-rest protection, but the OS may show an access prompt.
- **Master password** — secrets are encrypted with a password you choose. On each launch they start **locked**; enter the password once per session to unlock them (a locked secret shows an **Unlock** link where it's used). If you forget the password, the secrets **cannot be recovered**.

Existing installs keep using the OS keychain until you switch here; new installs default to **This device**. Switching to or from **OS keychain** may show one system prompt during the re-encryption.

Note: only secret *values* are encrypted. Execution history, response bodies, and the cookie jar are stored unencrypted in every mode.

Retries

Automatically retry failed requests with backoff. Configure the **max attempts**, the **backoff base**, **multiplier**, and **max delay**, and choose what to retry on — **connection errors**, **timeouts**, and/or a list of **status codes** (e.g. `429`, `503`, `504`). Retries are off by default.

When a retried response carries a **Retry-After** header, Rest Hippo waits for the time the server asks (capped at your **max delay**) instead of the computed backoff. To avoid duplicating a write, **connection-error and timeout** retries apply only to idempotent methods (GET, HEAD, PUT, DELETE, ...) — a

`POST / PATCH` that fails mid-flight might already have been processed, so it is **not** retried on a network error unless you enable **Retry POST/PATCH on network errors**. (Status-code retries still apply to every method, since the server's response shows it didn't act on the request.)

History

Set how many runs each request keeps in its [Timeline](#) (1–10).

Command Line

The **Command Line** panel installs a `hippo` command so you can launch Rest Hippo from a terminal — the equivalent of VS Code's "Install `code` command in PATH". Click **Install** and afterwards typing `hippo` in any terminal opens Rest Hippo (or focuses it if it's already running); click **Remove** to undo it.

The first time you start a freshly installed copy, Rest Hippo also offers to set this up for you — you can decline and install it later from here.

How it works per platform:

- **macOS / Linux** — a small launcher script is written to `/usr/local/bin` when that's writable, otherwise to `~/local/bin`. If the fallback location isn't already on your `PATH`, Rest Hippo tells you so you can add it.
- **Windows** — a `hippo.cmd` shim is created and its folder is added to your per-user `PATH`. Open a **new** terminal afterwards to pick up the change.

The command is only available in the installed app — it's disabled when running a development build, since there's no packaged executable to point at.

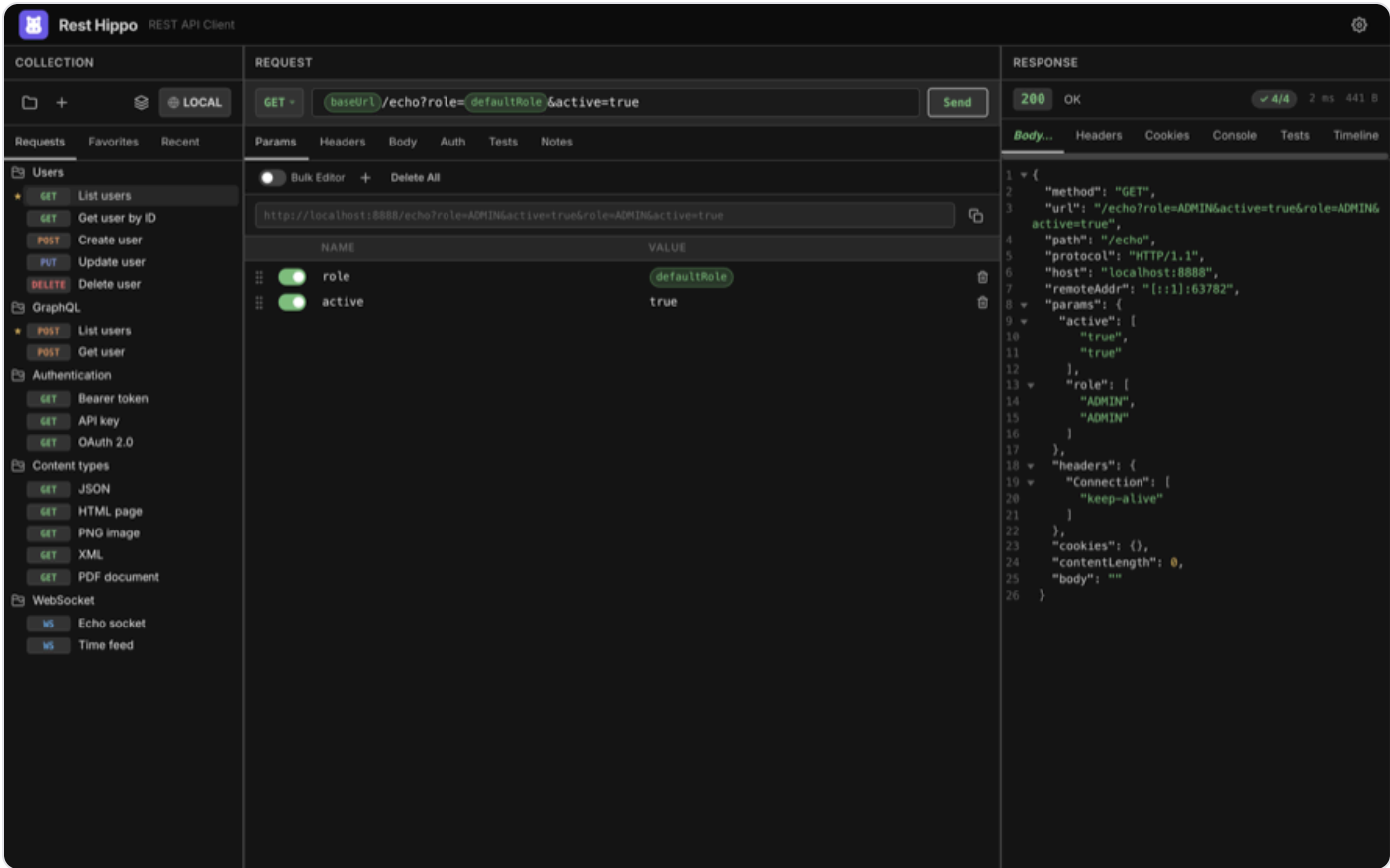
About & updates

The **About** panel shows the installed version, an **Automatically check for updates** toggle, and a **Check for Updates** button. Automatic checking is **off by default**: enable the toggle to have Rest Hippo look for new releases shortly after launch. The **Check for Updates** button — and **Help → Check for Updates...** — run on demand regardless of the toggle.

When a newer release is found it downloads in the background; once it's ready a notification offers to **Restart** and install it — Rest Hippo never restarts on its own. Updates are signed, so they install without security prompts.

Layouts

The **layout picker** in **Appearance** → **Layout** rearranges the three panels into four configurations — click a layout icon to switch:



Layout	Arrangement
Side by side	Collections Request Response, in three columns
Left + stacked	Collections on the left; Request above Response on the right
Top + full bottom	Collections and Request on top; Response full-width below
All stacked	The three panels stacked top to bottom

Rest Hippo also adapts automatically to narrow windows, and remembers where you drag the panel dividers.

Themes

Rest Hippo ships with four built-in themes — **Mocha** (the default dark theme), **Grey dark**, **Latte** (light), and **Grey light** — selectable from **Appearance → Theme**.

For full control, choose **Theme Editor...** from the theme dropdown to open the editor in its own window. It has two tabs:

- **Colors** — tune every colour token: backgrounds, text, accent and semantic colours, plus the per-method request badges.
- **Metrics** — adjust the layout tokens that control how dense the interface feels: the **spacing** scale, **control heights** (buttons, inputs, selects), corner **radii**, and the **splitter** thickness. Each is a pixel value.

Both tabs update the app with a **live preview**, so you can see your changes as you make them. Save your creation as a custom theme that appears in the theme list. Custom themes can be exported and imported to share.

The interface font for **context menus** always uses your OS's native typeface (San Francisco, Segoe UI, ...); the **UI font** setting controls everything else.

Next: [Keyboard Shortcuts →](#)

Keyboard Shortcuts

[← Back to contents](#)

On macOS use  (Command); on Windows/Linux use .

Global

These work anywhere in the app. Press  +  at any time to pop up a quick reference to the most common shortcuts inside Rest Hippo.

Shortcut	Action
 /  + 	Send the request (or WebSocket message when connected)
 /  + 	New request
 /  +  + 	New WebSocket request
 /  +  + 	New collection (or folder when a node is selected)
 /  + 	Focus the URL bar
 /  +  + 	Select the previous request
 /  +  + 	Select the next request
 /  +  /  / 	Switch to the Requests / Favorites / Recent tab
 /  + 	Cycle through the panel layouts
 /  + 	Open Settings
 /  + 	Open the active collection's environments
 /  +  + 	Open the Collections manager
 /  + 	Open the keyboard-shortcuts cheat-sheet
 /  + 	Open the User Guide
 /  +  + 	Import a collection

The navigation keys (focus URL, next/previous request, switch tab) yield to text entry — they only fire when you aren't typing in a field. The application menus also list these accelerators next to their commands.

There's no Save shortcut — you don't need one. Every edit you make to a request (its method, URL, params, headers, body, auth, ...) is **saved automatically** a moment after you stop typing. Your work is always persisted; there is no unsaved state to lose.

Sending requests

Shortcut	Action
	Send the request (while the URL bar is focused)
+	Send the request, or a WebSocket message when connected

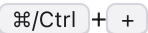
Response viewer

Shortcut	Action
+	Open the find-in-response bar
+ +	Open the filter bar (JSON / YAML / XML)
	Next match (in the find bar)
+	Previous match (in the find bar)
+	Select the entire response body
	Close the find or filter bar

Editors & typeahead

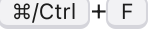
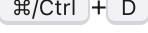
Shortcut	Action
	Open the variable typeahead
/	Move through autocomplete suggestions
/	Accept the highlighted suggestion
	Dismiss suggestions
+	Undo in the focused editor field
+ +	Redo in the focused editor field

Appearance

Shortcut	Action
	Increase font size
	Decrease font size
	Reset font size

Navigating the request tree

The collections tree is a single keyboard stop:  into it once, then move around entirely with the arrow keys — you don't tab through every row.

Shortcut	Action
	Move focus into the tree (lands on the selected request)
 / 	Move to the previous / next visible row
	Expand a collapsed collection, or step into an open one
	Collapse an open collection, or jump to its parent
 / 	Jump to the first / last visible row
<i>Type a name</i>	Type-ahead: jump to the next row that starts with it
	Reveal the filter box above the list
	Hide the filter box and clear the filter
	Open the request, or toggle the collection
	Select the row without opening / toggling
	Duplicate the focused request (requests only)
	Rename the focused request or folder
 / 	Delete the focused request or folder (opens a confirm)

The same  /  /  /  keys move between entries in the collections list.

Pickers & menus

Dropdown menus and selectors throughout the app are fully keyboard-operable:

Shortcut	Action
Enter / Space / ↓	Open the menu
↑ / ↓	Move between options
Home / End	First / last option
Enter / Space	Apply the highlighted option
Esc	Close without changing

Tree & dialogs

Shortcut	Action
Enter	Confirm an inline rename
Esc	Cancel a rename / close the current popup or dialog
Double-click	Rename a request, folder, or collection

Accessibility

Rest Hippo respects your operating system's accessibility preferences automatically — there's nothing to turn on in the app:

- **Reduce motion** — when your OS asks apps to minimize motion, Rest Hippo drops the slide-in, scale, and pulse animations on toasts, menus, and indicators (loading spinners keep turning, since the motion is the "in progress" signal).
- **Increased contrast / High Contrast** — muted secondary text is strengthened, and the focus ring and the current-selection outline stay visible (including in Windows High Contrast / forced-colors mode).

Every interactive control is reachable by keyboard and shows a focus ring when focused that way. Icon-only buttons carry text labels for screen readers, and status messages (errors, saves) are announced as they appear.

[← Back to contents](#)